UNIVERSIDAD EUROPEA

Grado en Ingeniería Informática · Programación con Estructuras Lineales

Actividad Colaborativa 2

Sistema de administración académico

Memoria del proyecto: gestión de recursos y memoria dinámica en C++

GRUPO 3

Adrián Duque

Carlota Parra

Ignacio del Peso

Sofía Peral

Índice

1. Introducción....3

2. Análisis del enunciado....4

2.1. Qué pide cada recurso y cómo lo resolvemos....4
2.2. Cómo hemos simplificado el sistema....5

3. Marco teórico....6

3.1. Abstracción y Tipos Abstractos de Datos....6
3.2. Programación orientada a objetos....6
3.3. Genericidad y plantillas....7
3.4. Gestión de recursos y ciclo de vida....7
3.5. Espacio de memoria de un proceso....7
3.6. Punteros y memoria dinámica....8
3.7. Paginación, alineación y localidad....9
3.8. Destructores, copia de objetos y regla de los tres....9
3.9. Excepciones y seguridad ante excepciones....10
3.10. RAII....10

4. Diseño....11

4.1. Diagrama de clases....11
4.2. Justificación de cada clase....13
4.3. Ficheros del proyecto....13

5. Implementación....14

5.1. Lista genérica ListaGenerica<T>....14
5.2. Usuario: clase base, polimorfismo e ids únicos....15
5.3. Recurso 1: Profesor y gestión de la clase....17
5.4. Recurso 2: Estudiante, notas y mochila de ficheros....19
5.5. Recurso 3: Matriz<T> y calculadora....21
5.6. Recurso 4: Administrador, altas y bajas....26
5.7. Programa principal: inicio de sesión, menús y excepciones....28
5.8. Resumen de la gestión de memoria....30

6. Manual de uso....31

6.1. Compilar y ejecutar....31
6.2. Inicio de sesión....31
6.3. Menú del administrador....32
6.4. Menú del profesor....33
6.5. Menú del estudiante....35
6.6. Calculadora....36
6.7. Salir y límites del programa....38

7. Pruebas....39

7.1. Casos de prueba....39
7.2. Errores controlados....39
7.3. Bajas y destructores....44
7.4. Prueba de memoria con AddressSanitizer....46

8. Reparto del trabajo....47

9. Conclusiones....48

9.1. Posibles mejoras....48

10. Referencias....49

1. Introducción

Esta memoria describe el **Sistema de administración académico** desarrollado por el Grupo 3 para la Actividad Colaborativa 2 de *Programación con Estructuras Lineales*. Se trata de una aplicación de consola en C++ para un centro de formación profesional, con la que estudiantes, profesores y administradores trabajan sobre cuatro recursos: la gestión de los estudiantes de cada aula, una mochila de material digital, una calculadora avanzada con matrices y un sistema de altas y bajas de usuarios.

La actividad sirve para practicar los contenidos de la **Unidad Formativa 2 (Gestión de recursos)**: el ciclo de vida de los objetos, la pila de ejecución frente al *free store*, la reserva y liberación de memoria con `new` y `delete`, los destructores, la propiedad de los recursos, las excepciones y la seguridad ante excepciones. También se apoya en la UF1 (tipos abstractos de datos y plantillas), porque todas las colecciones del programa se guardan en la lista genérica que se hizo en clase.

El documento está organizado así: el apartado 2 interpreta el enunciado y explica cómo hemos simplificado el sistema; el 3 resume la teoría de la asignatura que hemos usado; el 4 presenta el diseño de clases; el 5 explica la implementación de cada parte, enlazando cada concepto teórico con el código concreto y con el **porqué** de cada decisión; el 6 es el manual de uso con capturas de la ejecución real; el 7 recoge las pruebas; y los apartados 8 y 9 cierran con el reparto del trabajo y las conclusiones.

2. Análisis del enunciado

2.1. Qué pide cada recurso y cómo lo resolvemos

Antes de escribir código leímos el enunciado frase a frase para separar lo que se pide de lo que no. La tabla relaciona cada requisito con la solución y la clase o función que lo implementa.

Requisito del enunciado	Solución adoptada	Dónde
R1. Cada profesor almacena y visualiza los datos de sus estudiantes en un almacén personal	Cada profesor tiene su propia lista de punteros a estudiante	`Profesor::clase`
R1. Un profesor solo tiene una clase y un estudiante solo puede estar con un profesor	La clase es un atributo del profesor; el estudiante guarda un puntero a su profesor y no se puede inscribir si ya tiene uno	`Profesor::inscribir`, `Estudiante::profesor`
R1. Ver nombre, apellido y notas por asignatura; nº de inscritos y listado	Listado de la clase con los datos y notas de cada alumno	`Profesor::verClase`, `Estudiante::mostrarDatos`
R2. Mochila de documentos; un documento es un objeto fichero que lee un .txt del disco	Clase `Fichero` (ruta y contenido); la mochila es una lista de `Fichero*` creada en el free store	`Fichero`, `Estudiante::mochila`
R2. Cargar varios ficheros y ver su contenido	Menú del estudiante: cargar, ver y eliminar documentos	`cargarFichero`, `verMochila`, `quitarFichero`
R3. Calculadora con operaciones básicas	Suma, resta, multiplicación y división de números	funciones de `Main.cpp`
R3. Matrices de n elementos: suma, resta, multiplicación y división	Clase plantilla `Matriz<T>` hasta 10×10; la división es $A \cdot B^{-1}$	`Matriz<T>`
R3 (opcional). Traspuesta y determinante	Implementadas y accesibles desde el menú	`traspuesta`, `determinante`
R4. Rol Administrador que da de alta y de baja usuarios y gestiona nombre, apellido y número de usuario	Menú de administrador con alta, baja, modificación y listado	`Administrador`
R4. Id irrepetible	Cada usuario recibe el menor id libre; al darse de baja su id vuelve a quedar libre	`Usuario::idsEnUso`
R4. Los usuarios se eliminan de memoria al darse de baja	`delete` del usuario; los destructores deshacen sus relaciones	`darBaja`, destructores
Manejar todas las excepciones y la eficiencia	Validación de la entrada, `throw` en cada operación que puede fallar, `try/catch` en todos los menús y liberación garantizada de la memoria	todo el proyecto

2.2. Cómo hemos simplificado el sistema

La rúbrica premia simplificar el sistema para facilitar aún más las necesidades del usuario. Por eso solo hemos modelado los sustantivos del enunciado que tienen comportamiento propio y que pueden existir varias veces. Estas son las decisiones principales:

- **No hay clase Aula.** El enunciado dice que un profesor solo puede tener una clase, así que la clase es un atributo del profesor (`ListaGenerica<Estudiante*> clase`). Crear una clase Aula añadiría un objeto intermedio sin comportamiento propio.
- **No hay clase Mochila.** La mochila solo guarda documentos, los muestra y los quita: eso ya lo hace la lista genérica. Es una `ListaGenerica<Fichero*>` que pertenece al estudiante.
- **No hay clase Calculadora.** La calculadora no guarda estado y solo existiría un objeto, así que sus operaciones son funciones de `Main.cpp`. La clase que sí hace falta es `Matriz<T>`, porque hay muchas matrices y cada una tiene sus datos.
- **Documento = Fichero.** El enunciado lo dice literalmente: «un documento será un objeto fichero».
- **Inicio de sesión por número de usuario.** El enunciado no pide contraseñas, así que el usuario entra escribiendo su id, que es su número de usuario. Al arrancar se crea un administrador inicial con id 1 para poder dar de alta al resto.
- **La calculadora está disponible para los tres roles** (opción 9 de cada menú), porque el enunciado la presenta como una funcionalidad más del programa y no la asocia a ningún rol.
- **División de matrices.** No existe una división de matrices como tal; lo habitual es definir A / B como $A \cdot B^{-1}$, multiplicar por la inversa. Por eso B tiene que ser cuadrada e invertible, y el programa avisa si no lo es.

> **Resultado:** 7 clases (`Usuario`, `Estudiante`, `Profesor`, `Administrador`, `Fichero`, `ListaGenerica<T>` y `Matriz<T>`) y un `Main.cpp` con los menús. En el apartado 4.2 se justifica cada una con la frase del enunciado que la pide.

3. Marco teórico

En este apartado resumimos la teoría de la asignatura que hemos aplicado. Seguimos las diapositivas de la UF1 (*Tipos Abstractos de Datos y Principios de Diseño*), la UF2 (*Gestión de recursos*) y la *Guía para programar en C++*. Al final de cada punto indicamos dónde se usa en el proyecto; el detalle está en el apartado 5.

3.1. Abstracción y Tipos Abstractos de Datos

La **abstracción** consiste en quedarse con lo esencial de un problema e ignorar los detalles que no importan en ese momento. Separa dos niveles: **qué** hace un componente (su especificación) y **cómo** lo hace (su implementación). El código que usa una estructura debería depender solo de lo que la estructura hace, no de cómo lo hace por dentro.

Un **Tipo Abstracto de Datos (TAD)** define un conjunto de valores y las operaciones que se pueden hacer con ellos. Su **interfaz** son los métodos públicos y su **representación** son los atributos privados. Las operaciones se clasifican en **constructoras** (dejan el objeto en un estado inicial válido), **consultoras** (dan información sin modificar el estado) y **modificadoras** (cambian el estado). Cada operación tiene **precondiciones** (lo que debe cumplirse antes de llamarla) y **postcondiciones** (lo que se cumple después), y la estructura mantiene unos **invariantes** que nunca deben romperse.

En el proyecto: `ListaGenerica<T>` es un TAD lista. Su invariante es que las posiciones válidas van de 0 a `count-1` y que `count` está entre 0 y 50. La precondición de `at(posicion)` es que la posición sea válida; si no se cumple, lanza `out_of_range`.

3.2. Programación orientada a objetos

Una **clase** define un tipo de dato y un **objeto** es una instancia concreta con su propio estado. El **constructor** establece un estado inicial válido. La **encapsulación** agrupa datos y operaciones y oculta la representación: los atributos son privados y solo se modifican a través de métodos. Un método declarado `const` garantiza que no modifica el objeto, por lo que las consultoras deben ser `const`.

Para el **paso de parámetros**, los tipos primitivos (`int`, `char`, `bool`) se pasan por valor porque copiarlos cuesta poco. Los objetos grandes que solo se leen se pasan por **referencia constante** (`const T&`), que evita la copia. Una referencia no constante se usa cuando la función tiene que modificar el argumento.

En C++ cada clase se divide en un fichero de **interfaz** (`.h`), donde se declara lo que ofrece, y un fichero de **implementación** (`.cpp`), donde se define cómo lo hace.

La **herencia** crea una clase hija a partir de una clase padre, que hereda sus atributos y métodos. El **polimorfismo dinámico** permite que una misma llamada se comporte según el objeto real que la recibe. Se consigue declarando la función como `virtual` en la clase base y reescribiéndola con `override` en las derivadas (Guía de C++, apartado 10). Si un método no es virtual, al llamarlo a través de un puntero a la clase base se ejecuta siempre la versión de la base, sin importar el objeto real.

En el proyecto: `Usuario` es la clase base de `Estudiante`, `Profesor` y `Administrador`. `getRol()`, `mostrarDatos()` y el destructor son virtuales, porque todos los usuarios se guardan como `Usuario*` en la misma lista.

3.3. Genericidad y plantillas

La **genericidad** permite escribir una sola vez una función o una clase que funciona con tipos que no se conocen de antemano. En C++ se consigue con **plantillas** (`template <typename T>`): el compilador genera una versión concreta para cada tipo con el que se usa la plantilla (`ListaGenerica<int>`, `ListaGenerica<Usuario*>`…). Las plantillas reducen el **acoplamiento**, porque la estructura no depende del significado de lo que guarda.

Una plantilla impone **requisitos sobre el tipo**: si la implementación compara con `==` o copia elementos, el tipo `T` tiene que permitirlo. Estos requisitos no aparecen en la cabecera, pero forman parte del contrato y hay que documentarlos. Además, como el compilador genera el código en el sitio donde se usa la plantilla, necesita ver la implementación completa. Por eso las plantillas suelen escribirse enteras en el `.h`.

En el proyecto: `ListaGenerica<T>` (todo en el `.h`) y `Matriz<T>`, que tiene la implementación en el `.cpp` y usa instanciación explícita (apartado 5.5).

3.4. Gestión de recursos y ciclo de vida

Un **recurso** es cualquier elemento limitado que el programa adquiere y después debe liberar: memoria dinámica, ficheros, conexiones… La gestión de recursos responde a tres preguntas: **quién adquiere** el recurso, **quién es su propietario** (y por tanto quién lo libera) y **cuándo** se libera. Una mala gestión provoca fugas de memoria, accesos a zonas no válidas, liberaciones duplicadas o datos incoherentes. Son errores que a veces no se notan en las pruebas y aparecen más adelante.

Todo se apoya en el **ciclo de vida de los objetos**: creación (constructor), manipulación (métodos) y destrucción (destructor). El constructor deja el objeto en un estado válido y el destructor libera los recursos que le pertenecen.

En el proyecto: el apartado 5.8 tiene una tabla con cada `new` del programa, quién es su propietario y dónde se hace el `delete`.

3.5. Espacio de memoria de un proceso

Al ejecutar un programa, el sistema operativo crea un proceso con un **espacio de direcciones virtual**; el sistema operativo y la MMU lo traducen a memoria física. Ese espacio se divide en regiones:

Región	Qué contiene	En nuestro programa
`.text`	Instrucciones ejecutables	El código de todas las funciones y métodos
`.data` / `.bss`	Variables estáticas inicializadas / sin inicializar	`Usuario::idsEnUso`, `MAX_DIM`, `TOLERANCIA`
Almacenamiento libre (free store)	Memoria dinámica reservada con `new`; su tamaño varía durante la ejecución	Usuarios, mochilas, ficheros y matrices de la calculadora

Región	Qué contiene	En nuestro programa
Pila de usuario	Marcos de activación de las funciones: variables locales, argumentos y dirección de retorno	La lista `usuarios` de `main`, las opciones de los menús, el `ifstream` de `Fichero::cargar`

La **pila de ejecución** guarda los marcos de las funciones. Cuando `f()` llama a `g()`, se guarda la dirección de retorno y se crea un marco nuevo para `g()` con sus variables y argumentos; al terminar `g()`, el puntero de pila vuelve a su valor original y esas variables desaparecen.

El **free store** (memoria dinámica) sirve para datos cuyo tamaño o duración no se conocen al escribir el programa. Se reserva con `new` y se libera con `delete`. Un objeto dinámico **no se destruye al salir del bloque** en el que se declaró el puntero que lo señala: si se pierde el puntero sin hacer `delete`, el objeto se queda ocupando memoria. Eso es una **fuga de memoria**.

El **ámbito** indica desde dónde se puede usar un nombre y la **duración de almacenamiento** indica cuánto tiempo existe el objeto:

Duración	Variables afectadas	Fin de la vida	Ejemplo en el proyecto
Estática	Globales, de namespace o `static`	Al terminar el programa	`Usuario::idsEnUso`
Automática	Locales	Al salir del bloque, en orden inverso a su creación	`ListaGenerica<Usuario*> usuarios` en `main`
Dinámica	Creadas con `new`	Al hacer `delete`	`new Estudiante(...)` en `darAlta`

3.6. Punteros y memoria dinámica

Un **puntero** es una variable que guarda una dirección de memoria. `&x` obtiene la dirección de `x` y `*p` accede al valor apuntado. Un puntero solo debe usarse si tiene una dirección válida o `nullptr`, que indica que no apunta a nada. A diferencia de Java, donde las referencias son implícitas y existe un recolector de basura, en C++ el programador declara los punteros y gestiona la memoria a mano.

`new` reserva memoria y ejecuta el constructor; devuelve un puntero al objeto. `delete` ejecuta el destructor y devuelve la memoria, pero el puntero sigue existiendo y conserva una dirección que ya no es válida. Los arrays se reservan con `new[]` y se liberan con `delete[]`; mezclar las dos formas es comportamiento indefinido.

Los errores típicos de la gestión dinámica son:

- **Fuga de memoria:** se pierde el acceso a una memoria reservada antes de liberarla.
- **Doble liberación:** se libera dos veces el mismo recurso.
- **Puntero colgante:** un puntero conserva la dirección de un objeto que ya se ha destruido; usarlo es un error.
- **Acceso fuera de límites:** un array no sabe qué posiciones son válidas desde el punto de vista lógico; salirse de ellas modifica memoria ajena.

En el proyecto: los cuatro errores se evitan de forma explícita. Las listas comprueban los límites. Los destructores de `Estudiante` y `Profesor` eliminan los punteros colgantes entre ellos. Cada `new` tiene un único propietario que hace el `delete` correspondiente. Las capturas de AddressSanitizer del apartado 7.4 demuestran que no hay fugas.

3.7. Paginación, alineación y localidad

La **paginación** divide la memoria virtual en páginas del mismo tamaño que el sistema operativo coloca en marcos de la memoria física. Gracias a ella, varios procesos pueden ejecutarse a la vez sin ver la memoria de los demás. Es transparente para nuestro programa.

La **alineación** obliga a colocar cada dato en una dirección múltiplo de su tamaño (4 para `int`, 8 para punteros y `double`). Para cumplirla, el compilador añade bytes de relleno (*padding*) entre atributos, y el tamaño de un objeto acaba siendo múltiplo de la alineación de su miembro más grande. Lo hemos comprobado con `sizeof` en nuestras clases:

Tipo	sizeof	Explicación
`Usuario`	80 bytes	puntero a la tabla virtual (8) + `int id` (4) + **4 de relleno** + 2 `string` (32 + 32)
`ListaGenerica<int>`	204 bytes	50 × 4 + `int count` (4): todo alineado a 4, sin relleno
`ListaGenerica<Usuario*>`	408 bytes	50 × 8 + `count` (4) + **4 de relleno** para que el tamaño sea múltiplo de 8
`Profesor`	488 bytes	`Usuario` (80) + `ListaGenerica<Estudiante*>` (408)
`Estudiante`	1 912 bytes	`Usuario` (80) + asignaturas (1 608) + notas (204) + **4 de relleno** + 2 punteros (16)
`Matriz<double>`	808 bytes	100 × 8 + 2 `int` (8)

La **localidad** explica por qué la caché funciona. Hay localidad **espacial** cuando, tras acceder a una dirección, es probable acceder enseguida a una cercana, y localidad **temporal** cuando es probable volver a la misma dirección. Por eso los datos contiguos que se recorren en orden son mucho más rápidos.

En el proyecto: `ListaGenerica` guarda sus elementos en un array contiguo (`almacen[50]`). `Matriz` también (`datos[10][10]`), y todos sus bucles recorren por filas, con el índice de fila fuera y el de columna dentro. Así se respeta el orden en que el array está en memoria y se aprovecha la localidad espacial.

3.8. Destructores, copia de objetos y regla de los tres

Un **destructor** (`~Clase()`) no recibe parámetros, no devuelve nada y solo hay uno por clase. Si la clase posee memoria dinámica u otros recursos, el destructor debe liberarlos. Cuando muere un objeto se ejecuta **primero el cuerpo de su destructor** y **después se destruyen sus atributos**, en orden inverso al de construcción; en una clase derivada se destruye por último la parte de la clase base. Los objetos automáticos se destruyen al salir de su bloque, los dinámicos con `delete` y los estáticos al terminar el programa.

La copia que genera el compilador copia los atributos uno a uno. Si un atributo es un puntero, se copia la **dirección**, no lo apuntado (**copia superficial**). En una clase propietaria esto es un error:

cuando uno de los dos objetos libera el recurso, el otro se queda con un puntero colgante, y cuando lo libere habrá una doble liberación. La **copia profunda** crea un recurso nuevo para la copia. De ahí sale la **regla de los tres**: si una clase gestiona un recurso y necesita definir el destructor, el constructor de copia o el operador de asignación, normalmente necesita definir los tres. La **regla de los cinco** añade el constructor y la asignación por movimiento.

En el proyecto: `Estudiante` es dueño de su mochila. Como nunca hace falta copiar un estudiante (siempre se maneja con `Estudiante*`), en lugar de programar una copia profunda que nadie usaría hemos **prohibido la copia** (`= delete`). Así cualquier intento de copiarlo da un error de compilación en vez de una doble liberación en ejecución.

3.9. Excepciones y seguridad ante excepciones

Un **error de ejecución** aparece con el programa funcionando. Las **excepciones** permiten comunicar el error por un camino distinto del resultado normal: la función que no puede terminar su tarea hace `throw` y el código que la llamó lo trata con `try`/`catch`. La biblioteca estándar distingue dos familias:

- `std::logic_error` (`invalid_argument`, `out_of_range`, `domain_error`…): errores que se podrían detectar antes de ejecutar, como un argumento incorrecto o una posición fuera de rango.
- `std::runtime_error`: errores difíciles de prever, debidos a causas externas al programa, como un fichero que no existe.

Los `catch` se examinan en orden y se ejecuta el primero compatible. `catch (...)` captura cualquier excepción y suele usarse para **limpiar y relanzar** con `throw;`. Cada operación que puede fallar debe ofrecer una **garantía**:

- **Básica:** no hay fugas y el objeto sigue siendo válido, aunque haya cambiado.
- **Fuerte:** la operación se completa o no cambia nada.
- **No lanzamiento:** la operación nunca lanza excepciones. Es imprescindible en los destructores.

En el proyecto: el apartado 5.7 tiene una tabla con todas las excepciones que se lanzan y dónde se capturan. `darAlta` y `cargarFichero` ofrecen la garantía fuerte y ningún destructor lanza excepciones.

3.10. RAII

RAII (*Resource Acquisition Is Initialization*) asocia cada recurso a la vida de un objeto: se adquiere en el constructor y se libera en el destructor. Así la liberación no depende de acordarse de escribir `delete` en cada salida de una función, porque el destructor se ejecuta siempre, también cuando salta una excepción.

En el proyecto: el `std::ifstream` de `Fichero::cargar` cierra el fichero en su destructor al salir de la función, aunque haya error. `Estudiante` aplica la misma idea a su mochila: la reserva en el constructor y la libera en el destructor. En la calculadora, donde los punteros son locales, el `catch (...)` hace la limpieza a mano.

4. Diseño

4.1. Diagrama de clases

La Figura 1, en la página siguiente, muestra el diagrama UML del proyecto actualizado con la versión final del código.

Las relaciones del diagrama son:

- **Herencia:** `Estudiante`, `Profesor` y `Administrador` heredan de `Usuario`.
- **Composición `main` → `Usuario`:** el programa principal es dueño de todos los usuarios y los libera al salir.
- **Composición `Estudiante` → `Fichero`:** los documentos de la mochila pertenecen al estudiante y se destruyen con él.
- **Agregación `Profesor` ◇ `Estudiante`:** el profesor referencia a sus alumnos con punteros, pero no es su dueño. Si el profesor desaparece, los alumnos siguen existiendo sin clase.
- **Dependencias de uso:** las clases usan `ListaGenerica<T>`, la calculadora usa `Matriz<T>` y el administrador gestiona la lista de usuarios.

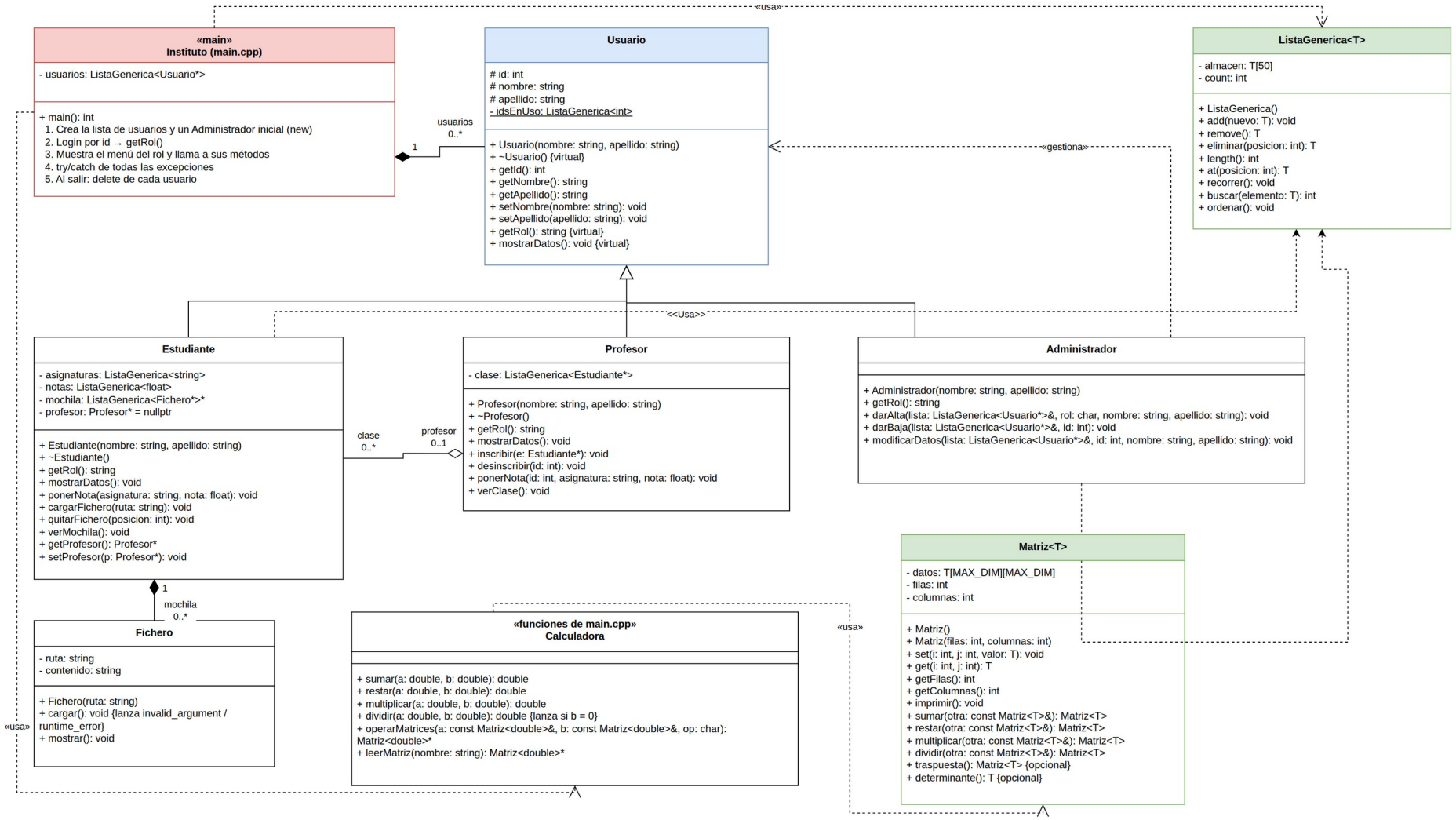


Figura 1. *Diagrama de clases UML del proyecto (fichero `Diagrama UML.drawio`).*

4.2. Justificación de cada clase

Clase	Frase del enunciado que la justifica	Por qué es una clase
`Usuario`	«los usuarios (estudiantes o profesores)…», «número de usuario», «id irrepetible»	Agrupa lo común a los tres roles y permite guardarlos juntos en una lista
`Estudiante`	«rol de Estudiante… debe poder tener una mochila»	Hay muchos estudiantes, cada uno con sus notas y su mochila
`Profesor`	«cada profesor debe poder almacenar y visualizar los datos de sus estudiantes»	Hay varios profesores, cada uno con su clase
`Administrador`	«El sistema tendrá un rol adicional. El Administrador»	Es un rol más; sus operaciones son altas, bajas y modificación
`Fichero`	«Un documento será un objeto fichero… que leerá del disco un fichero.txt»	El enunciado lo pide literalmente; cada fichero tiene su ruta y su contenido
`ListaGenerica<T>`	«almacén personal», «mochila», listado de usuarios	Es el contenedor genérico de clase; evita repetir arrays con contadores en cada clase
`Matriz<T>`	«operar con matrices de n elementos»	Cada matriz tiene sus dimensiones y datos, y las operaciones son suyas

4.3. Ficheros del proyecto

Fichero	Contenido
`ListaGenerica.h`	Plantilla de la lista genérica (declaración e implementación)
`Usuario.h / .cpp`	Clase base, ids únicos y reutilización de ids libres
`Estudiante.h / .cpp`	Notas por asignatura y mochila de ficheros
`Profesor.h / .cpp`	Clase del profesor: inscribir, desinscribir, notas y listado
`Administrador.h / .cpp`	Altas, bajas y modificación de usuarios
`Fichero.h / .cpp`	Documento de la mochila: lectura de un .txt
`Matriz.h / .cpp`	Plantilla de matriz y sus operaciones
`Main.cpp`	Inicio de sesión, menús de cada rol y calculadora
`CMakeLists.txt`	Configuración de compilación; copia la carpeta `ficheros` junto al ejecutable
`ficheros/`	Documentos .txt de ejemplo para la mochila
`Diagrama UML.drawio`	Diagrama de clases

5. Implementación

En cada parte se indica quién la desarrolló, se muestran capturas del código real con sus números de línea y se explica **por qué** se ha usado cada técnica en ese sitio.

5.1. Lista genérica ListaGenerica<T>

> *Responsable: Carlota Parra (issue #1, base del proyecto).*

Todas las colecciones del programa usan la misma estructura: la lista genérica que hicimos en el ejercicio de clase *Lista Genérica*. Es una clase plantilla con un array de tamaño fijo y un contador de posiciones ocupadas (Figura 2).

ListaGenerica.h (líneas 8–38)

```
 8 // Contenedor generico (TAD lista) que usan todas las colecciones del proyecto:
 9 // usuarios, alumnos de una clase, asignaturas, notas, mochila e ids en uso.
10 // Es una plantilla, asi que el cuerpo de los metodos tiene que estar en el .h:
11 // el compilador genera el codigo para cada tipo concreto (ListaGenerica<int>,
12 // ListaGenerica<Usuario*>...) en el sitio donde se usa.
13 //
14 // Invariante: las posiciones validas son 0..count-1 y count esta entre 0 y 50.
15 // Requisitos sobre T: tener constructor por defecto y poder copiarse (array interno),
16 // == para buscar y << para recorrer.
17 template <typename T>
18 class ListaGenerica {
19 private:
20     T almacen[50];   // tamaño fijo: los elementos quedan contiguos en memoria
21     int count;       // numero de posiciones ocupadas
22 public:
23     // Constructora: deja la lista en un estado valido (vacia)
24     ListaGenerica() {
25         count = 0;
26     }
27
28     // Modificadora: añade al final. Lanza out_of_range si la lista esta llena
29     void add(T nuevo) {
30         if (count >= 50) {
31             throw std::out_of_range("La lista esta llena (maximo 50 elementos).");
32         }
33         almacen[count] = nuevo;
34         count++;
35     }
36
37     // Modificadora: saca y devuelve el ultimo elemento
38     T remove() {
```

Figura 2. *ListaGenerica.h: representación, constructora y operaciones add/remove.*

Por qué así

- **Plantilla en el .h:** el compilador genera una versión distinta para cada tipo con el que la usamos (`int`, `float`, `std::string`, `Usuario*`, `Estudiante*` y `Fichero*`) y necesita ver el cuerpo de los métodos en cada sitio donde se instancia. Por eso está entera en la cabecera.
- **Una sola estructura para todo:** sin ella, cada clase tendría su propio array con su contador, repetido y con los mismos errores posibles. Esto es el bajo acoplamiento de la UF1: la lista no sabe qué guarda.
- **Array contiguo de 50 posiciones:** los elementos están seguidos en memoria (localidad espacial) y la lista no necesita memoria dinámica propia, así que no hay nada que liberar.

Cuando la lista guarda punteros, la propiedad de lo apuntado es de quien la usa (apartado 5.8).

- **Operaciones clasificadas como en la UF1:** constructora (`ListaGenerica()`), modificadoras (`add`, `remove`, `eliminar`) y consultoras `const` (`length`, `at`, `buscar`).
- **Precondiciones comprobadas:** si la lista está llena o la posición no existe se lanza `out_of_range`, en vez de escribir fuera del array (acceso fuera de límites).

ListaGenerica.h (líneas 46-72)

```
46      // Eliminar en una posición específica desplazando a la izquierda
47      T eliminar(int posicion) {
48          if (posicion < 0 || posicion >= count) {
49              throw std::out_of_range("Posicion no valida para eliminar.");
50          }
51          T elementoEliminado = almacen[posicion];
52          for (int i = posicion; i < count - 1; i++) {
53              almacen[i] = almacen[i + 1];
54          }
55          count--;
56          return elementoEliminado;
57      }
58
59      // Consultoras (const): no modifican la lista
60      int length() const {
61          return count;
62      }
63
64      // Consulta sin modificar la lista
65      T at(int position) const {
66          if (position < 0 || position >= count) {
67              throw std::out_of_range("Indice fuera de rango");
68          }
69          return almacen[position];
70      }
71
72      void recorrer() const {
```

Figura 3. *ListaGenerica.h: eliminar en una posición, consultoras y búsqueda lineal.*

`eliminar(posicion)` desplaza los elementos siguientes una posición a la izquierda para mantener el invariante: las posiciones ocupadas siempre son las primeras. Devuelve el elemento quitado para que, si es un puntero, quien llama pueda liberarlo. `buscar` es la búsqueda lineal genérica de la UF1, que exige que `T` se pueda comparar con `==`.

5.2. Usuario: clase base, polimorfismo e ids únicos

Responsable: Carlota Parra (issue #1). La reutilización de ids se añadió en la revisión final (issue #8).

Usuario.h (líneas 7–31)

```
// Clase base de los tres roles (Estudiante, Profesor y Administrador).
// Guarda lo que tienen en comun: id, nombre y apellido.
// Los atributos son protected para que las clases hijas puedan usarlos.
class Usuario {
protected:
    int id;
    std::string nombre;
    std::string apellido;
    static ListaGenerica<int> idsEnUso; // Ids ocupados: al dar de baja se libera y se reutiliza
public:
    Usuario(std::string nombre, std::string apellido);
    // Virtual: al hacer delete sobre un Usuario* se ejecuta tambien el destructor de la clase hija
    virtual ~Usuario();

    int getId() const;
    std::string getNombre() const;
    std::string getApellido() const;

    void setNombre(std::string nombre);
    void setApellido(std::string apellido);

    // Virtual: con un Usuario* se ejecuta la version del objeto real (polimorfismo dinamico)
    virtual std::string getRol();
    virtual void mostrarDatos();
};
```

Figura 4. *Usuario.h: atributos comunes, destructor virtual y métodos virtuales.*

Polimorfismo dinámico

Todos los usuarios se guardan en una sola lista de `Usuario*`. Cuando alguien inicia sesión, el programa llama a `u->getRol()` para saber qué menú mostrar. Si `getRol()` no fuera `virtual`, esa llamada ejecutaría siempre la versión de `Usuario` y nunca se abriría ningún menú. Al declararla `virtual` en la base y `override` en las hijas, se ejecuta la versión del objeto real: el polimorfismo dinámico de la Guía de C++. `override` además hace que el compilador avise si la firma de la hija no coincide con la de la base.

El **destructor también es virtual**. Los usuarios se liberan con `delete` sobre un `Usuario*`. Sin destructor virtual solo se ejecutaría `~Usuario()`: no se liberaría la mochila del estudiante, el profesor no soltaría a sus alumnos y, además, el comportamiento sería indefinido. Con él, `delete` ejecuta `~Estudiante()` o `~Profesor()` y después `~Usuario()`.

Los atributos son `protected` y no `private` porque las clases hijas los usan directamente, por ejemplo `Estudiante::verMochila` muestra `nombre` y `apellido`.

Id irrepetible con reutilización de huecos

Usuario.cpp (líneas 4–18)

```
ListaGenerica<int> Usuario::idsEnUso;

Usuario::Usuario(std::string nombre, std::string apellido) {
    // Se asigna el menor id libre (empezando en 1, el 0 es "salir"), asi se reutilizan los de las bajas.
    // Si ya hay 50 usuarios, add lanza out_of_range y el objeto no llega a crearse.
    int candidato = 1;
    while (idsEnUso.buscar(candidato) != -1) {
        candidato++;
    }
    idsEnUso.add(candidato);

    this->id = candidato;
    this->nombre = nombre;
    this->apellido = apellido;
}
```

Figura 5. *Usuario.cpp: el constructor asigna el menor id libre.*

El enunciado pide que cada usuario tenga un id irrepetible, y la rúbrica pide además que, cuando alguien se da de baja, su id vuelva a usarse. Para eso `Usuario` tiene una lista **estática** con los

ids en uso. Al ser `static`, existe una sola para toda la clase, vive en el segmento de datos y dura todo el programa (duración estática). El constructor busca el menor número libre a partir de 1 (el 0 está reservado para «salir» en el inicio de sesión) y lo apunta en la lista.

Usuario.cpp (líneas 20–26)

```
20 Usuario::~Usuario() {
21     // Al destruir el usuario su id queda libre para la siguiente alta
22     int pos = idsEnUso.buscar(id);
23     if (pos != -1) {
24         idsEnUso.eliminar(pos);
25     }
26 }
```

Figura 6. *Usuario.cpp: el destructor libera el id.*

El id se libera en el **destructor**, es decir, la baja se hace destruyendo el objeto, como pide la rúbrica. El destructor solo usa `buscar` y elimina el id si lo encuentra, así que **nunca lanza** (garantía de no lanzamiento). Si ya hay 50 usuarios, `idsEnUso.add` lanza `out_of_range` dentro del constructor: el objeto no llega a crearse y `new` devuelve la memoria automáticamente, sin fuga.

5.3. Recurso 1: Profesor y gestión de la clase

Responsable: Carlota Parra (issue #3).

Profesor.h (líneas 8–29)

```
 8 class Estudiante;
 9
10 // Un profesor tiene una unica clase: la lista de sus estudiantes ("almacen personal").
11 class Profesor : public Usuario {
12 private:
13     ListaGenerica<Estudiante*> clase; // Punteros: el profesor no es dueño de los estudiantes
14
15 public:
16     // Constructor
17     Profesor(std::string nombre, std::string apellido);
18     ~Profesor() override;
19
20     // Métodos heredados de Usuario
21     std::string getRol() override;
22     void mostrarDatos() override;
23
24     // Métodos de profesor
25     void inscribir(Estudiante* e);
26     void desinscribir(int id);
27     void ponerNota(int id, std::string asignatura, float nota);
28     void verClase();
29 };
```

Figura 7. *Profesor.h: la clase del profesor es una lista de punteros a estudiante.*

La clase del profesor es una `ListaGenerica<Estudiante*>`. Se guardan **punteros** y no objetos porque el profesor no es dueño de sus alumnos: los estudiantes viven en la lista general de usuarios y solo se referencian. Guardarlos por valor crearía copias y, al poner una nota, se modificaría la copia y no el estudiante real.

`Profesor.h` declara `class Estudiante;` en vez de incluir `Estudiante.h` (**forward declaration**). Como solo se usan punteros, basta con saber que el tipo existe. Además se evita la inclusión circular, porque `Estudiante` también apunta a `Profesor`.

Profesor.cpp (líneas 8–40)

```cpp
Profesor::~Profesor() {
    // El profesor no es dueño de sus estudiantes: no se borran, solo se quedan sin clase
    for (int i = 0; i < clase.length(); i++) {
        clase.at(i)->setProfesor(nullptr);
    }
}

std::string Profesor::getRol() {
    return "Profesor";
}

void Profesor::mostrarDatos() {
    std::cout << "[Profesor] "<< std::endl;
    Usuario::mostrarDatos();
}

// inscribir y desinscribir actualizan siempre los dos lados de la relacion:
// la clase del profesor y el puntero "profesor" del estudiante. Asi se cumple que
// un estudiante esta en una clase si y solo si su profesor no es nullptr.
void Profesor::inscribir(Estudiante* e) {
    if (e == nullptr) {
        throw std::invalid_argument("El estudiante no puede ser nulo.");
    }
    // Comprobar si ya tiene profesor asignado
    if (e->getProfesor() != nullptr) {
        throw std::runtime_error("El estudiante ya esta inscrito en otra clase.");
    }

    clase.add(e);
    e->setProfesor(this);
}

void Profesor::desinscribir(int id) {
```

Figura 8. *Profesor.cpp: destructor e inscripción.*

`inscribir` y `desinscribir` mantienen el **invariante** de la relación: un estudiante está en una clase si y solo si su puntero `profesor` no es `nullptr`. Por eso `inscribir` rechaza a un alumno que ya tiene profesor, cumpliendo «un estudiante solo puede estar asociado a un profesor». El destructor del profesor pone a `nullptr` el puntero de cada alumno: si no, los alumnos se quedarían apuntando a un profesor destruido (**puntero colgante**). No los borra, porque no son suyos.

Profesor.cpp (líneas 40–56)

```cpp
void Profesor::desinscribir(int id) {
    bool encontrado = false;
    for (int i = 0; i < clase.length(); i++) {
        Estudiante* e = clase.at(i);
        if (e->getId() == id) {
            e->setProfesor(nullptr);
            clase.eliminar(i);
            encontrado = true;
            break;
        }
    }
    if (!encontrado) {
        throw std::out_of_range("Estudiante no encontrado en esta clase.");
    }
}

void Profesor::ponerNota(int id, std::string asignatura, float nota) {
```

Figura 9. *Profesor.cpp: desinscribir y poner nota.*

`ponerNota` busca al alumno en la clase y le delega la nota. La validación de que esté entre 0 y 10 la hace `Estudiante`, que es quien guarda las notas. `verClase` muestra el número de inscritos y, para cada alumno, su nombre, apellido y notas por asignatura, que es exactamente lo que pide el recurso 1 (Figura 36).

5.4. Recurso 2: Estudiante, notas y mochila de ficheros

> *Responsable: Adrián Duque (issue #4).*

Estudiante.h (líneas 18–58)

```
18 class Estudiante : public Usuario {
19 private:
20     // Listas paralelas: la nota de asignaturas.at(i) es notas.at(i).
21     ListaGenerica<std::string> asignaturas;
22     ListaGenerica<float> notas;
23 
24     // La mochila y cada fichero se crean con new (free store); el estudiante es su dueño
25     // y los libera en su destructor.
26     ListaGenerica<Fichero*>* mochila;
27 
28     // Por puntero: el profesor vive en la lista de usuarios, aqui solo se referencia.
29     Profesor* profesor;
30 
31 public:
32     Estudiante(const std::string& nombre, const std::string& apellido);
33     ~Estudiante() override;
34 
35     // Regla de los tres: como el estudiante es dueño de memoria dinamica (la mochila),
36     // una copia automatica copiaria solo el puntero y los dos objetos acabarian
37     // liberando la misma mochila (doble liberacion). Los estudiantes siempre se manejan
38     // con Estudiante*, asi que no hace falta copiarlos: se prohibe la copia.
39     Estudiante(const Estudiante&) = delete;
40     Estudiante& operator=(const Estudiante&) = delete;
41 
42     std::string getRol() override;
43     void mostrarDatos() override;
44 
45     // Si la asignatura ya existe actualiza su nota; si no, la añade.
46     // Lanza invalid_argument si la nota no esta entre 0 y 10.
47     void ponerNota(const std::string& asignatura, float nota);
48 
49     // Lanza runtime_error si el fichero no se puede abrir.
50     void cargarFichero(const std::string& ruta);
51     // posicion empieza en 1, como se muestra en verMochila.
52     // Lanza out_of_range si la posicion no existe.
53     void quitarFichero(int posicion);
54     void verMochila();
55 
56     Profesor* getProfesor() const;
57     void setProfesor(Profesor* p);
58 };
```

Figura 10. *Estudiante.h: notas, mochila en el free store y regla de los tres.*

Notas por asignatura

Las notas se guardan en dos listas paralelas: `asignaturas` y `notas`, donde la nota de `asignaturas.at(i)` es `notas.at(i)`. Si se vuelve a poner nota en una asignatura que ya existe, `ponerNota` quita la entrada de las dos listas a la vez y la añade de nuevo, para que sigan emparejadas. La nota se valida antes de tocar nada (`invalid_argument` si no está entre 0 y 10), así que si falla no se modifica nada.

La mochila vive en el free store

Estudiante.cpp (líneas 10–29)

```
10 Estudiante::Estudiante(const std::string& nombre, const std::string& apellido)
11     : Usuario(nombre, apellido) {
12     profesor = nullptr; // Al darse de alta todavia no esta en ninguna clase
13     mochila = new ListaGenerica<Fichero*>();
14 }
15
16 Estudiante::~Estudiante() {
17     // Si esta en una clase, se quita de ella para que el profesor no se quede
18     // con un puntero a un estudiante que ya no existe (puntero colgante).
19     // profesor != nullptr solo cuando esta inscrito, asi que desinscribir siempre lo encuentra.
20     if (profesor != nullptr) {
21         profesor->desinscribir(id);
22     }
23
24     // Se libera cada fichero y despues la propia mochila
25     while (mochila->length() > 0) {
26         delete mochila->remove();
27     }
28     delete mochila;
29 }
```

Figura 11. *Estudiante.cpp: el constructor reserva la mochila y el destructor la libera.*

La mochila es un puntero a una lista de `Fichero*`. Se reserva con `new` en el constructor y se libera en el destructor, siguiendo la idea de RAII: el estudiante **adquiere** la mochila al nacer y la **libera** al morir. Cada documento también se crea con `new`, porque el número de ficheros que va a cargar cada alumno no se conoce hasta la ejecución, que es justo el caso de uso de la memoria dinámica.

Esto también mejora la eficiencia. Antes, la mochila era una lista de 50 objetos `Fichero` dentro de cada estudiante: unos 5 KB por alumno aunque no cargase nada. Ahora el estudiante solo guarda un puntero de 8 bytes a la mochila y cada fichero ocupa memoria solo cuando se carga (`sizeof(Estudiante)` pasa de unos 5 100 a 1 912 bytes).

El destructor sigue el orden de la UF2. Primero, si el alumno está en una clase, se quita de ella, para que el profesor no se quede con un puntero colgante. Después libera cada fichero y, por último, la propia mochila. Al terminar su cuerpo, se destruyen los atributos y se ejecuta `~Usuario()`, que libera el id.

Cargar y eliminar documentos

Estudiante.cpp (líneas 65–81)

```
65 void Estudiante::cargarFichero(const std::string& ruta) {
66     Fichero* nuevo = new Fichero(ruta);
67     try {
68         nuevo->cargar();    // Si falla lanza la excepcion y no se añade nada a la mochila
69         mochila->add(nuevo);
70     } catch (...) {
71         delete nuevo;       // no se ha guardado: se libera para que no haya fuga
72         throw;
73     }
74 }
75
76 void Estudiante::quitarFichero(int posicion) {
77     Fichero* quitado = mochila->eliminar(posicion - 1);
78     delete quitado;
79 }
80
81 void Estudiante::verMochila() {
```

Figura 12. *Estudiante.cpp: cargar un fichero con garantía fuerte y eliminarlo.*

`cargarFichero` ofrece la **garantía fuerte**: o el documento se carga y se añade, o la mochila queda como estaba. Si `cargar()` lanza (el fichero no existe o no es .txt) o la mochila está llena, el `catch (...)` libera el `Fichero` recién creado y relanza la excepción para que el menú muestre el error. Sin ese `delete` habría una fuga cada vez que se intentase cargar un fichero que no existe.

`quitarFichero` saca el puntero de la lista y hace `delete`. Primero se quita de la lista y después se libera, para que la lista nunca contenga la dirección de un objeto destruido.

Regla de los tres

Como `Estudiante` es dueño de memoria dinámica, la copia automática sería superficial: dos estudiantes compartirían la misma mochila y los dos la liberarían (doble liberación). Por eso se prohíben el constructor de copia y la asignación con `= delete` (Figura 10). No hace falta una copia profunda porque los estudiantes siempre se manejan con punteros.

Fichero

Fichero.cpp (líneas 15–36)

```
15 void Fichero::cargar() {
16     // El enunciado pide ficheros .txt (asi tampoco se puede cargar una carpeta)
17     const std::string extension = ".txt";
18     if (ruta.length() <= extension.length() ||
19         ruta.compare(ruta.length() - extension.length(), extension.length(), extension) != 0) {
20         throw std::invalid_argument("Solo se pueden cargar ficheros .txt: " + ruta);
21     }
22
23     std::ifstream entrada(ruta);
24     if (!entrada.is_open()) {
25         throw std::runtime_error("No se ha podido abrir el fichero: " + ruta);
26     }
27
28     contenido = "";
29     std::string linea;
30     while (std::getline(entrada, linea)) {
31         contenido += linea + "\n";
32     }
33     // El ifstream se cierra solo al salir de la funcion (su destructor libera el fichero).
34 }
35
36 void Fichero::mostrar() const {
```

Figura 13. *Fichero.cpp: lectura del .txt.*

`Fichero` guarda la ruta y el texto leído. `cargar()` comprueba primero que la ruta termine en `.txt`, como dice el enunciado (`invalid_argument`); así tampoco se puede cargar una carpeta. Después abre el fichero y lanza `runtime_error` si no puede: es un error externo al programa, que es justo para lo que sirve `runtime_error`. El `ifstream` es un ejemplo de **RAII**: no hace falta cerrarlo a mano, porque su destructor lo cierra al salir de la función, también si se ha lanzado una excepción.

5.5. Recurso 3: Matriz<T> y calculadora

> *Responsable: Ignacio del Peso (issue #5). La traspuesta y el determinante se conectaron al menú y las matrices pasaron al free store en la revisión final (issue #8).*

Matriz.h (líneas 8–36)

```
8  const int MAX_DIM = 10; // tamaño maximo de filas y columnas
9
10 // Matriz generica de hasta MAX_DIM x MAX_DIM. Las operaciones devuelven una matriz nueva
11 // y lanzan invalid_argument si las dimensiones no son compatibles.
12 // El cuerpo de la plantilla esta en Matriz.cpp; al final de ese fichero se indican los
13 // tipos con los que se puede usar (instanciacion explicita).
14 template <typename T>
15 class Matriz {
16 private:
17     T datos[MAX_DIM][MAX_DIM]; // contiguo y recorrido por filas (localidad espacial)
18     int filas;
19     int columnas;
20
21 public:
22     Matriz();
23     Matriz(int filas, int columnas);
24
25     void set(int i, int j, T valor);
26     T get(int i, int j) const;
27     int getFilas() const;
28     int getColumnas() const;
29     void imprimir() const;
30     Matriz<T> sumar(const Matriz<T>& otra) const;
31     Matriz<T> restar(const Matriz<T>& otra) const;
32     Matriz<T> multiplicar(const Matriz<T>& otra) const;
33     Matriz<T> dividir(const Matriz<T>& otra) const;
34     Matriz<T> traspuesta() const;
35     T determinante() const;
36 };
```

Figura 14. *Matriz.h: plantilla de matriz.*

`Matriz<T>` es una clase plantilla, igual que en el ejercicio de clase *MatrizGenerica*. Guarda los datos en un array bidimensional de tamaño máximo 10×10 y las dimensiones reales en `filas` y `columnas`. El constructor valida las dimensiones y lanza `invalid_argument` si no están entre 1 y 10: así ninguna matriz puede existir en un estado inválido.

A diferencia de `ListaGenerica`, la implementación está en `Matriz.cpp` y al final del fichero se indican los tipos con los que se puede usar (**instanciación explícita**, Figura 15). Así el `.h` queda como interfaz limpia, como en el resto de clases, y el compilador genera el código de `Matriz<int>`, `Matriz<float>` y `Matriz<double>` una sola vez. La contrapartida es que solo se puede usar con esos tipos; la calculadora usa `Matriz<double>`.

Matriz.cpp (líneas 221–224)

```
221 // Tipos con los que se puede usar Matriz<T> (al estar la implementacion en el .cpp)
222 template class Matriz<int>;
223 template class Matriz<float>;
224 template class Matriz<double>;
```

Figura 15. *Matriz.cpp: instanciación explícita de la plantilla.*

Matriz.cpp (líneas 61–75)

```
61 Matriz<T> Matriz<T>::sumar(const Matriz<T>& otra) const {
62     if (filas != otra.filas || columnas != otra.columnas) {
63         throw std::invalid_argument("Las matrices deben tener las mismas dimensiones");
64     }
65     Matriz<T> resultado(filas, columnas);
66     for (int i = 0; i < filas; i++) {
67         for (int j = 0; j < columnas; j++) {
68             resultado.datos[i][j] = datos[i][j] + otra.datos[i][j];
69         }
70     }
71     return resultado;
72 }
73
74 template <typename T>
75 Matriz<T> Matriz<T>::restar(const Matriz<T>& otra) const {
```

Figura 16. *Matriz.cpp: suma con comprobación de dimensiones.*

Cada operación comprueba primero que las dimensiones sean compatibles y lanza `invalid_argument` si no lo son, antes de crear el resultado. Reciben la otra matriz por **referencia constante** (`const Matriz<T>&`), porque una matriz ocupa 808 bytes y solo hace falta leerla. Además son métodos `const`: no modifican la matriz sobre la que se llaman. Los bucles recorren por filas, aprovechando la localidad espacial.

División de matrices

Matriz.cpp (líneas 108–135)

```
108 Matriz<T> Matriz<T>::dividir(const Matriz<T>& otra) const {
109     if (otra.filas != otra.columnas) {
110         throw std::invalid_argument("El divisor debe ser una matriz cuadrada");
111     }
112     if (columnas != otra.filas) {
113         throw std::invalid_argument("Dimensiones incompatibles para dividir");
114     }
115     const int n = otra.filas;
116
117     // Gauss-Jordan sobre [otra | I] para obtener la inversa
118     double a[MAX_DIM][2 * MAX_DIM];
119     for (int i = 0; i < n; i++) {
120         for (int j = 0; j < n; j++) {
121             a[i][j] = otra.datos[i][j];
122             a[i][n + j] = (i == j) ? 1.0 : 0.0;
123         }
124     }
125     for (int c = 0; c < n; c++) {
126         // Pivoteo parcial: se usa la fila con el valor mas grande en esta columna.
127         // Con decimales un 0 puede quedar como 1e-17, por eso se compara con una tolerancia.
128         int pivote = c;
129         for (int i = c + 1; i < n; i++) {
130             if (std::fabs(a[i][c]) > std::fabs(a[pivote][c])) {
131                 pivote = i;
132             }
133         }
134         if (std::fabs(a[pivote][c]) < TOLERANCIA) {
135             throw std::invalid_argument("La matriz divisor no es invertible");
```

Figura 17. *Matriz.cpp: inversa por Gauss-Jordan con pivoteo parcial.*

A / B se calcula como $A \cdot B^{-1}$. La inversa se obtiene por Gauss-Jordan sobre la matriz ampliada [B | I]. En cada columna se elige como pivote la fila con el valor absoluto más grande (**pivoteo parcial**), que reduce los errores de redondeo. Si el mejor pivote es prácticamente cero (menor que `TOLERANCIA = 1e-9`), la matriz no es invertible y se lanza una excepción. Hace falta una tolerancia porque con `double` un cero real puede quedar como $6 \cdot 10^{-18}$ tras varias restas, y una comparación con `== 0` no lo detectaría. El determinante usa la misma eliminación y devuelve 0 en ese caso.

Calculadora y matrices en el free store

La calculadora son funciones de `Main.cpp` (apartado 2.2). Las operaciones básicas reciben y devuelven `double` por valor, porque son tipos primitivos. Dividir entre cero lanza una excepción en lugar de devolver infinito.

Main.cpp (líneas 34–44)

```
34 // El resultado se guarda en el free store; quien llama debe liberarlo con delete
35 Matriz<double> *operarMatrices(const Matriz<double> &a, const Matriz<double> &b, char op) {
36     switch (op) {
37         case '+': return new Matriz<double>(a.sumar(b));
38         case '-': return new Matriz<double>(a.restar(b));
39         case '*': return new Matriz<double>(a.multiplicar(b));
40         case '/': return new Matriz<double>(a.dividir(b));
41         default:
42             throw std::invalid_argument("Operacion no valida");
43     }
44 }
```

Figura 18. *Main.cpp: el resultado de operar se crea en el free store.*

Main.cpp (líneas 105–123)

```
105 // La matriz se crea en el free store; quien llama debe liberarla con delete
106 Matriz<double> *leerMatriz(string nombre) {
107     cout << "Matriz " << nombre << endl;
108     int filas = leerEntero("Numero de filas: ");
109     int columnas = leerEntero("Numero de columnas: ");
110
111     // Si las dimensiones no son validas el constructor lanza y new libera la memoria solo
112     Matriz<double> *m = new Matriz<double>(filas, columnas);
113
114     for (int i = 0; i < filas; i++) {
115         for (int j = 0; j < columnas; j++) {
116             cout << nombre << "[" << i << "][" << j << "] = ";
117             double valor = leerDecimal("");
118             m->set(i, j, valor);
119         }
120     }
121
122     return m;
123 }
```

Figura 19. *Main.cpp: las matrices que introduce el usuario se crean en el free store.*

Las matrices que introduce el usuario y la matriz resultado se crean con `new`. Su tamaño depende de lo que el usuario escriba en tiempo de ejecución y, al vivir en el free store, no ocupan la pila de las funciones del menú (cada una son 808 bytes). Lo importante es garantizar que **siempre** se liberan:

Main.cpp (líneas 174–215)

```cpp
                case 5: {
                    // Se elige la operacion antes de pedir las matrices
                    cout << "Operacion: 1 = Suma, 2 = Resta, 3 = Multiplicacion, 4 = Division" << endl;
                    int tipo = leerEntero("Operacion: ");

                    char op;
                    if (tipo == 1) {
                        op = '+';
                    } else if (tipo == 2) {
                        op = '-';
                    } else if (tipo == 3) {
                        op = '*';
                    } else if (tipo == 4) {
                        op = '/';
                    } else {
                        throw std::invalid_argument("Operacion no valida");
                    }

                    // Matrices y resultado en el free store. Si algo lanza una excepcion
                    // se libera lo que ya se habia creado antes de relanzarla.
                    Matriz<double> *a = nullptr;
                    Matriz<double> *b = nullptr;
                    Matriz<double> *resultado = nullptr;
                    try {
                        a = leerMatriz("A");
                        b = leerMatriz("B");
                        resultado = operarMatrices(*a, *b, op);
                        cout << "Resultado:" << endl;
                        resultado->imprimir();
                    } catch (...) {
                        delete resultado;
                        delete b;
                        delete a;
                        throw;
                    }
                    delete resultado;
                    delete b;
                    delete a;
                    break;
                }

                case 6: {
```

Figura 20. *Main.cpp: operación con matrices; se libera la memoria tanto si todo va bien como si salta una excepción.*

Los tres punteros empiezan en `nullptr`. Si algo falla (dimensiones incompatibles, matriz no invertible…), el `catch (...)` libera lo que se haya creado y relanza la excepción para que el menú la muestre. `delete` sobre `nullptr` no hace nada, así que no importa en qué punto haya fallado. Si todo va bien, se liberan al terminar. La operación se pide **antes** que las matrices, para no obligar al usuario a escribir dos matrices si la operación no es válida.

Main.cpp (líneas 215–248)

```
215                 case 6: {
216                     Matriz<double> *a = nullptr;
217                     Matriz<double> *traspuesta = nullptr;
218                     try {
219                         a = leerMatriz("A");
220                         traspuesta = new Matriz<double>(a->traspuesta());
221                         cout << "Traspuesta:" << endl;
222                         traspuesta->imprimir();
223                     } catch (...) {
224                         delete traspuesta;
225                         delete a;
226                         throw;
227                     }
228                     delete traspuesta;
229                     delete a;
230                     break;
231                 }
232 
233                 case 7: {
234                     Matriz<double> *a = nullptr;
235                     try {
236                         a = leerMatriz("A");
237                         // Lanza invalid_argument si la matriz no es cuadrada
238                         double determinante = a->determinante();
239                         cout << "Determinante: " << determinante << endl;
240                     } catch (...) {
241                         delete a;
242                         throw;
243                     }
244                     delete a;
245                     break;
246                 }
247 
248                 case 0:
```

Figura 21. *Main.cpp: traspuesta y determinante (opcionales del enunciado).*

5.6. Recurso 4: Administrador, altas y bajas

> *Responsable: Ignacio del Peso (issue #2). La baja mediante destructores se completó en la revisión final (issue #8).*

Administrador.cpp (líneas 13–56)

```cpp
void Administrador::darAlta(ListaGenerica<Usuario*>& lista, char rol,
                            const std::string& nombre, const std::string& apellido) {
    // Se crea en el free store para que el usuario siga existiendo al salir del metodo;
    // la lista solo guarda su direccion
    Usuario* nuevo = nullptr;

    if (rol == 'E') {
        nuevo = new Estudiante(nombre, apellido);
    } else if (rol == 'P') {
        nuevo = new Profesor(nombre, apellido);
    } else if (rol == 'A') {
        nuevo = new Administrador(nombre, apellido);
    } else {
        throw std::runtime_error("Rol no valido");
    }

    try {
        lista.add(nuevo);
    } catch (...) {
        delete nuevo;  // lista llena: no queda fuga
        throw;
    }
}

void Administrador::darBaja(ListaGenerica<Usuario*>& lista, int id) {
    int pos = -1;
    for (int i = 0; i < lista.length(); i++) {
        if (lista.at(i)->getId() == id) {
            pos = i;
            break;
        }
    }
    if (pos == -1) {
        throw std::runtime_error("El id no existe");
    }

    // Se saca de la lista y se elimina de memoria. Al ser el destructor virtual, delete ejecuta
    // el de la clase real: ~Estudiante o ~Profesor deshacen la relacion profesor-alumno
    // y despues ~Usuario deja libre el id.
    Usuario* u = lista.eliminar(pos);
    delete u;
}

void Administrador::modificarDatos(ListaGenerica<Usuario*>& lista, int id,
```

Figura 22. *Administrador.cpp: alta y baja de usuarios.*

La lista de usuarios no pertenece al administrador, sino a `main`. Por eso los métodos la reciben **por referencia** (`ListaGenerica<Usuario*>&`): así trabajan sobre la lista original sin copiarla (408 bytes).

`darAlta` crea el usuario con `new` según el rol, porque tiene que seguir existiendo cuando termine el método. Si la lista está llena, `add` lanza una excepción; el `catch (...)` libera el usuario recién creado y la relanza, así que no hay fuga y la lista no cambia (**garantía fuerte**). El id lo asigna el constructor de `Usuario`, que reutiliza los huecos.

`darBaja` es muy corta gracias a los destructores virtuales. Primero saca el puntero de la lista, para que nunca contenga un objeto destruido, y después hace `delete`. Ese `delete` ejecuta automáticamente:

- `~Estudiante()`: lo quita de la clase de su profesor y libera su mochila y sus ficheros.
- `~Profesor()`: deja a sus alumnos sin profesor (`nullptr`), para que puedan inscribirse con otro.
- `~Usuario()`: libera el id para la siguiente alta.

Así «los usuarios deberán eliminarse de memoria cuando se den de baja» se cumple **mediante el destructor del objeto usuario**, como pide la rúbrica, y no hace falta preguntar el tipo con `static_cast` ni comparar cadenas.

5.7. Programa principal: inicio de sesión, menús y excepciones

Responsable: Sofía Peral (issue #6).

Main.cpp (líneas 495-507)

```
495 int main() {
496     // Lista de todos los usuarios. Guarda punteros (Usuario*) para poder tener
497     // estudiantes, profesores y administradores en la misma lista (polimorfismo).
498     // La lista vive en la pila de main; los usuarios, en el free store.
499     ListaGenerica<Usuario *> usuarios;
500 
501     // Primer administrador, para poder entrar y dar de alta al resto (recibe el id 1)
502     Administrador *admin = new Administrador("Admin", "Principal");
503     usuarios.add(admin);
504 
505     cout << "Administrador creado. Su id es: " << admin->getId() << endl;
506 
507     bool salir = false;
```

Figura 23. *Main.cpp: la lista de usuarios y el administrador inicial.*

La lista `usuarios` es una **variable automática** de `main`: vive en la pila durante todo el programa. Los usuarios a los que apunta están en el free store. `main` es su propietario y los libera al salir.

Main.cpp (líneas 523-550)

```
523                 // getRol es virtual: devuelve el rol del objeto real. Con el rol ya se sabe
524                 // que tipo es, y static_cast convierte el Usuario* al puntero de su clase.
525                 string rol = u->getRol();
526 
527                 if (rol == "Administrador") {
528                     Administrador *a = static_cast<Administrador *>(u);
529                     menuAdministrador(a, usuarios);
530                 } else if (rol == "Profesor") {
531                     Profesor *p = static_cast<Profesor *>(u);
532                     menuProfesor(p, usuarios);
533                 } else if (rol == "Estudiante") {
534                     Estudiante *e = static_cast<Estudiante *>(u);
535                     menuEstudiante(e);
536                 }
537             }
538         }
539     }
540 
541     // Liberar la memoria de todos los usuarios creados con new.
542     // delete ejecuta el destructor de cada clase (virtual) y no queda ninguna fuga.
543     while (usuarios.length() > 0) {
544         Usuario *u = usuarios.remove();
545         delete u;
546     }
547 
548     cout << "Memoria liberada" << endl;
549 
550     return 0;
```

Figura 24. *Main.cpp: reparto por rol y liberación final de la memoria.*

Cuando alguien inicia sesión, `getRol()` (virtual) dice qué tipo es el objeto real. Con esa información, `static_cast` convierte el `Usuario*` al puntero de su clase para llamar a su menú. La conversión es segura porque se hace después de comprobar el rol. Al salir se recorre la lista haciendo `delete` de cada usuario; gracias a los destructores virtuales se libera también todo lo que cada uno posee, y AddressSanitizer confirma que no queda ninguna fuga (apartado 7.4).

Lectura validada de la entrada

Main.cpp (líneas 46–65)

```cpp
// ---- Lectura validada de datos ----
// Si el usuario escribe letras donde se espera un numero, cin queda en estado de error:
// se limpia (clear), se descarta la linea (ignore) y se vuelve a pedir.
int leerEntero(string mensaje) {
    int numero;
    cout << mensaje;
    cin >> numero;

    while (cin.fail()) {
        cin.clear();
        cin.ignore(10000, '\n');
        cout << "Entrada no valida. Escribe un numero: ";
        cin >> numero;
    }

    cin.ignore(10000, '\n');
    return numero;
}

double leerDecimal(string mensaje) {
```

Figura 25. *Main.cpp: lectura validada de números.*

Si el usuario escribe letras donde se espera un número, `cin` queda en estado de error y cualquier lectura posterior fallaría. `leerEntero` y `leerDecimal` detectan el fallo, limpian el estado (`clear`), descartan la línea (`ignore`) y vuelven a pedir el dato. `leerTexto` no acepta cadenas vacías. Así se previenen errores de ejecución sin necesidad de excepciones, porque es un error esperable del usuario.

Tratamiento de las excepciones

Cada menú envuelve la opción elegida en un `try` con un `catch (const exception& e)` que muestra el mensaje y vuelve al menú. Como todas las excepciones estándar heredan de `std::exception`, un solo `catch` las trata todas, y el programa nunca se cierra por un error. Se capturan por **referencia constante** para no copiar la excepción y conservar su tipo real. Esta tabla resume todas las excepciones del programa:

Dónde se lanza	Tipo	Cuándo
`ListaGenerica::add`	`out_of_range`	La lista está llena (50 elementos)
`ListaGenerica::at`, `remove`, `eliminar`	`out_of_range`	Posición no válida o lista vacía
`Usuario::Usuario`	`out_of_range`	Ya hay 50 ids en uso
`Administrador::darAlta`	`runtime_error`	Rol no válido
`Administrador::darBaja`, `modificarDatos`	`runtime_error`	El id no existe
`Profesor::inscribir`	`invalid_argument` / `runtime_error`	Estudiante nulo / ya tiene profesor
`Profesor::desinscribir`, `ponerNota`	`out_of_range`	El alumno no está en la clase
`Estudiante::ponerNota`	`invalid_argument`	Nota fuera de 0–10
`Estudiante::quitarFichero`	`out_of_range`	Número de documento no válido

Dónde se lanza	Tipo	Cuándo
`Fichero::cargar`	`invalid_argument` / `runtime_error`	No es un .txt / no se puede abrir
`Matriz` (constructor, operaciones)	`invalid_argument`	Dimensiones no válidas o incompatibles, matriz no invertible, determinante de matriz no cuadrada
`Matriz::set`, `get`	`out_of_range`	Posición fuera de la matriz
`dividir` (números)	`runtime_error`	División entre cero
`operarMatrices` / menú de matrices	`invalid_argument`	Operación no válida

Los `catch (...) { …; throw; }` de `darAlta`, `cargarFichero` y la calculadora no tratan el error: solo **liberan la memoria y relanzan** la excepción, como en el ejemplo de la UF2. El mensaje lo muestra el `catch` del menú.

5.8. Resumen de la gestión de memoria

Esta tabla responde a las tres preguntas de la gestión de recursos de la UF2 (quién adquiere, quién es el propietario y cuándo se libera) para cada `new` del programa:

Objeto	Se crea en	Propietario	Se libera en
Administrador inicial	`main`	`main`	Bucle final de `main`
Cada usuario	`Administrador::darAlta`	`main` (lista `usuarios`)	`darBaja` o al salir de `main`
Mochila de un estudiante	`Estudiante::Estudiante`	El estudiante	`~Estudiante`
Cada fichero	`Estudiante::cargarFichero`	El estudiante (mochila)	`quitarFichero`, `~Estudiante` o el `catch` si falla la carga
Matrices A, B y resultado	`leerMatriz`, `operarMatrices`, opciones 6 y 7	La opción del menú	Al acabar la opción o en su `catch`

Los punteros que no son propietarios (`Profesor::clase` y `Estudiante::profesor`) nunca hacen `delete`. De que no queden colgantes se encargan los destructores (apartados 5.3 y 5.4). Cada `new` tiene **un único propietario y un único `delete`**, lo que evita a la vez las fugas y la doble liberación.

6. Manual de uso

6.1. Compilar y ejecutar

El proyecto usa CMake y se abre directamente con **CLion**:

- Abrir la carpeta del proyecto en CLion (*File → Open*). CLion detecta el `CMakeLists.txt` y configura el proyecto.
- Compilar con *Build → Build Project* (Ctrl+F9) y ejecutar con *Run* (Mayús+F10). El programa se abre en la consola de CLion.
- Al compilar se copia la carpeta `ficheros` junto al ejecutable (en `cmake-build-debug`), así que los ficheros de ejemplo se cargan escribiendo, por ejemplo, `ficheros/actividad1.txt`.

También se puede compilar desde una terminal (Figura 26):

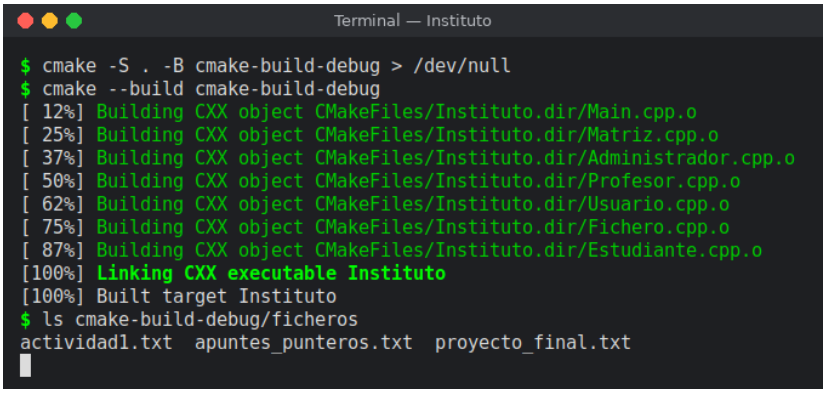


Figura 26. *Compilación con CMake; la carpeta ficheros se copia junto al ejecutable.*

6.2. Inicio de sesión

Al arrancar se crea el administrador principal con **id 1**. Para entrar se escribe el número de usuario; con `0` se sale del programa. Según el rol del usuario se abre su menú (Figura 27).

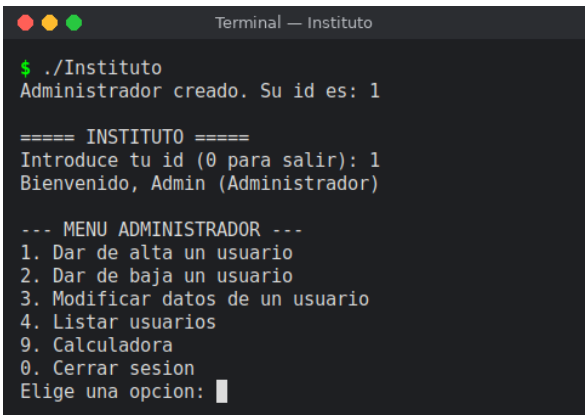


Figura 27. *Inicio del programa y acceso como administrador.*

6.3. Menú del administrador

Opción	Qué hace
1. Dar de alta un usuario	Pide el tipo (1 Estudiante, 2 Profesor, 3 Administrador), el nombre y el apellido, y muestra el id asignado
2. Dar de baja un usuario	Pide el id y elimina al usuario de memoria. El administrador no puede darse de baja a sí mismo
3. Modificar datos	Cambia el nombre y el apellido de un usuario
4. Listar usuarios	Muestra el id, el nombre y el rol de todos los usuarios
9. Calculadora	Abre la calculadora
0. Cerrar sesión	Vuelve a la pantalla de inicio de sesión

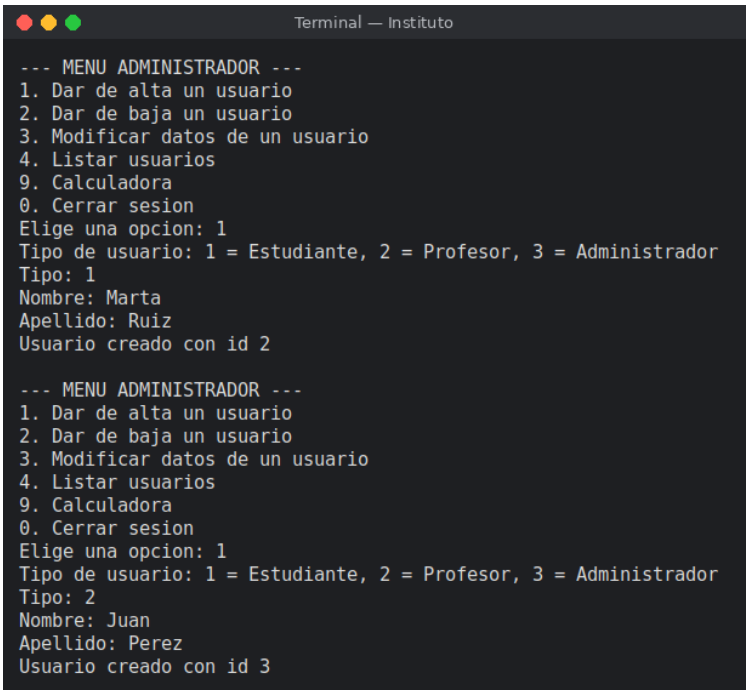


Figura 28. *Alta de una estudiante (id 2) y un profesor (id 3).*

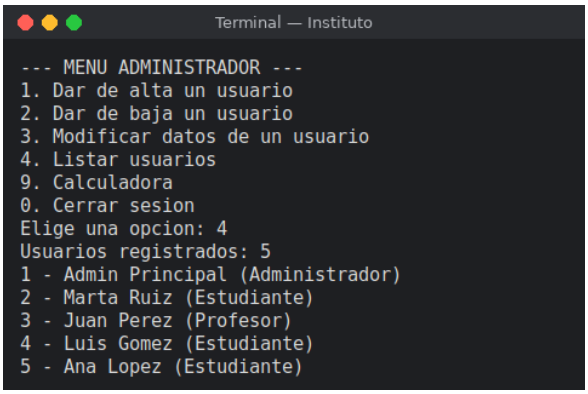


Figura 29. *Listado de usuarios tras dar de alta a cuatro personas.*

Al dar de baja a un usuario su id queda libre y el siguiente usuario que se dé de alta lo reutiliza. En la Figura 30 se da de baja al usuario 4 y el nuevo usuario recibe otra vez el id 4 (Figura 31).

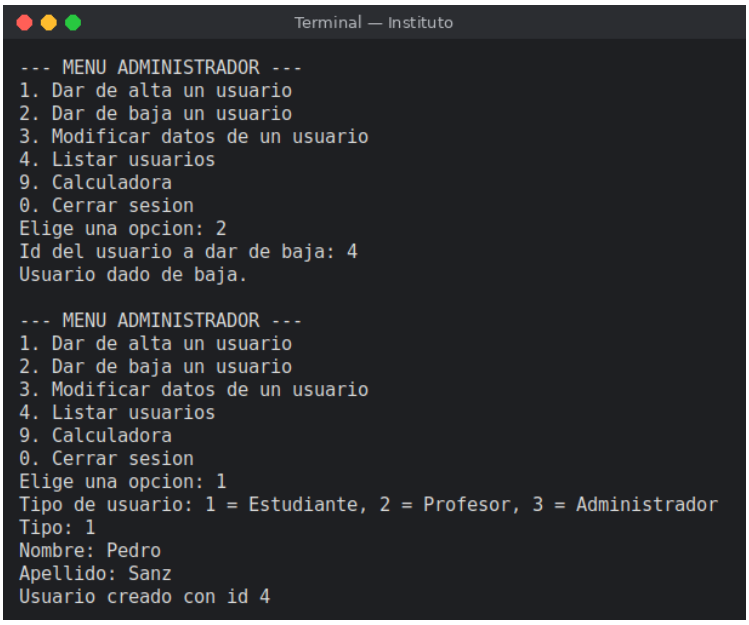


Figura 30. *Baja del usuario 4 y alta de un nuevo estudiante, que recibe el id 4.*

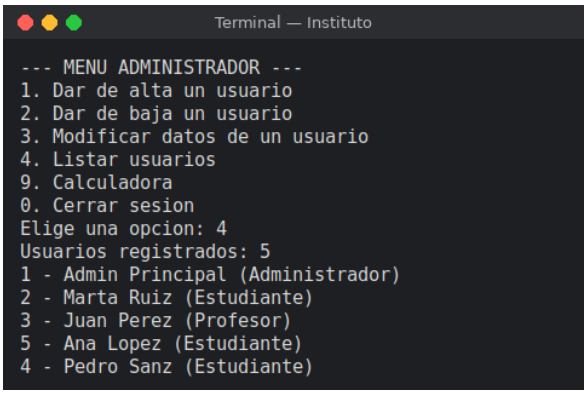


Figura 31. *Listado: el id 4 se ha reutilizado.*

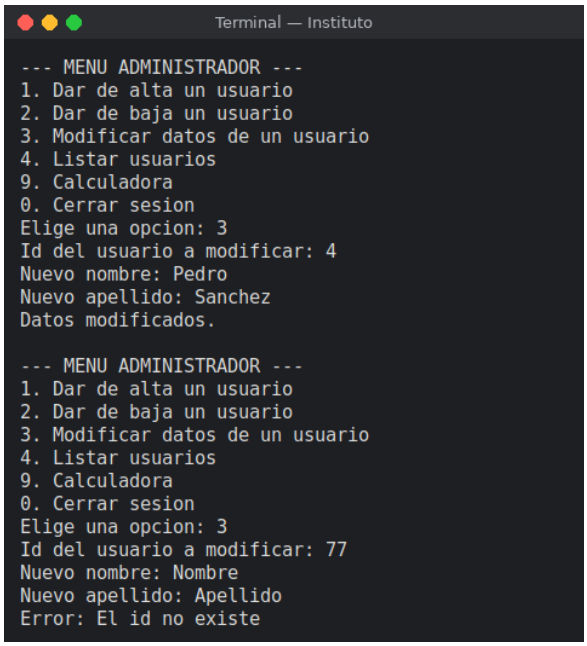


Figura 32. *Modificación de datos y error con un id que no existe.*

6.4. Menú del profesor

Opción	Qué hace
1. Inscribir un estudiante	Pide el id de un estudiante sin profesor y lo añade a la clase
2. Quitar un estudiante	Saca al estudiante de la clase (sigue existiendo como usuario)
3. Poner una nota	Pide el id, la asignatura y la nota (0 a 10). Si la asignatura ya tenía nota, se actualiza

Opción	Qué hace
4. Ver mi clase	Muestra el número de inscritos y el nombre, el apellido y las notas de cada alumno
9. Calculadora / 0. Cerrar sesión	Igual que en el resto de menús

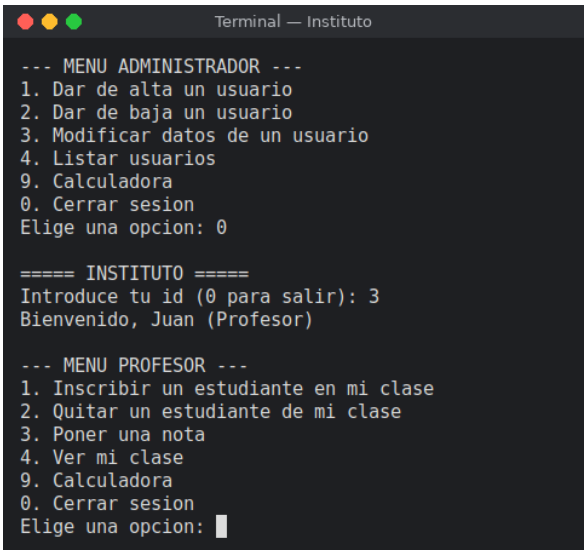


Figura 33. *Acceso como profesor.*

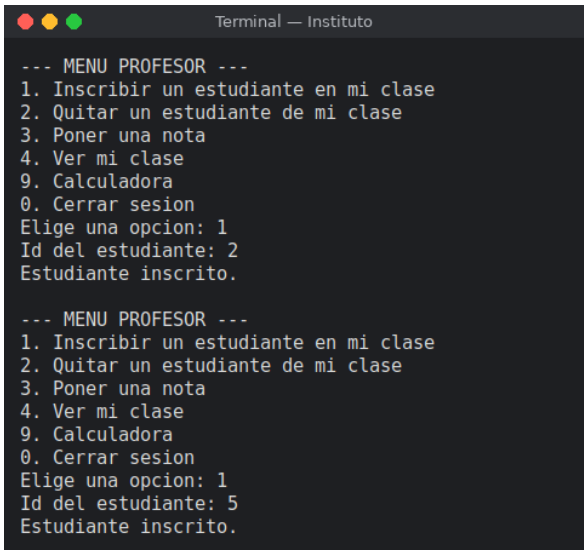


Figura 34. *Inscripción de dos estudiantes en la clase.*

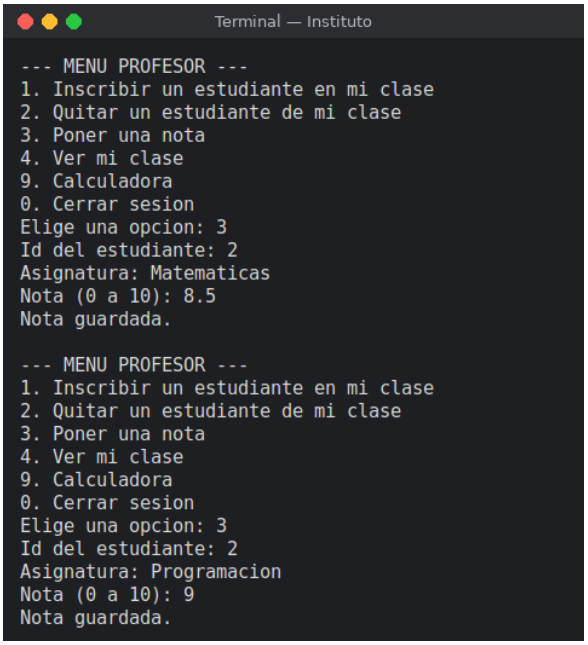


Figura 35. *Notas de dos asignaturas a una alumna.*

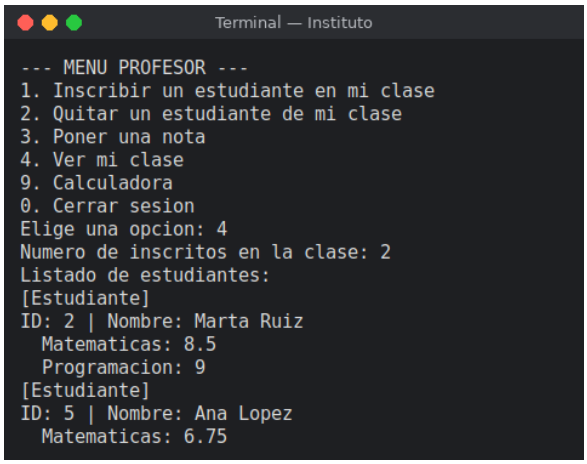


Figura 36. *Ver la clase: número de inscritos y listado con las notas.*

6.5. Menú del estudiante

Opción	Qué hace
1. Cargar un fichero	Pide la ruta de un .txt, lo lee y lo guarda en la mochila
2. Ver mi mochila	Muestra cada documento numerado con su contenido
3. Eliminar un fichero	Muestra la mochila y pide el número del documento a quitar
4. Ver mis datos y notas	Muestra el id, el nombre y las notas por asignatura
9. Calculadora / 0. Cerrar sesión	Igual que en el resto de menús

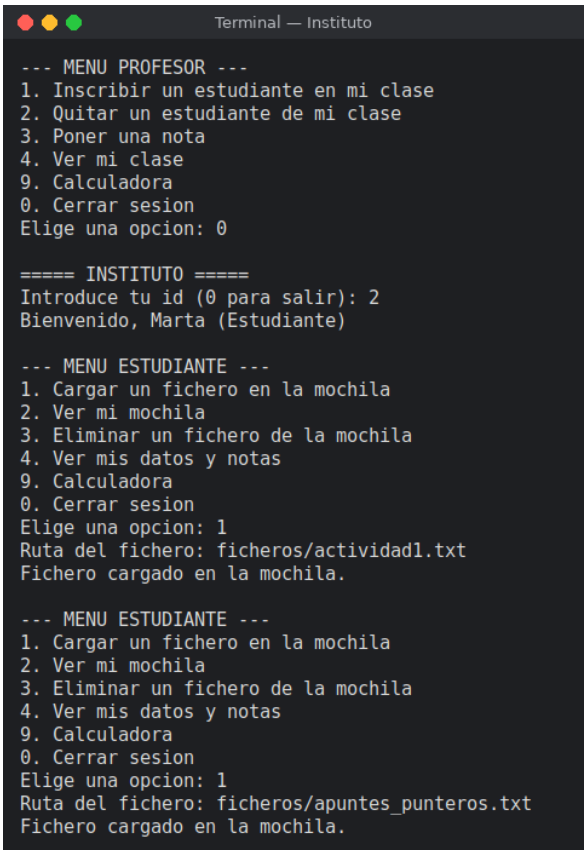


Figura 37. *Acceso como estudiante y carga de dos ficheros en la mochila.*

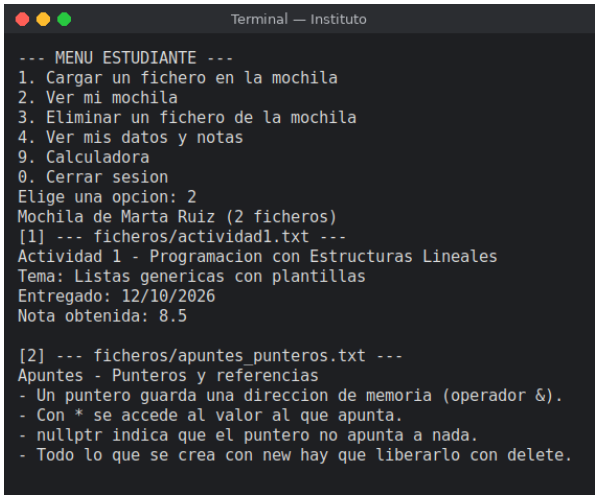


Figura 38. *Contenido de la mochila.*

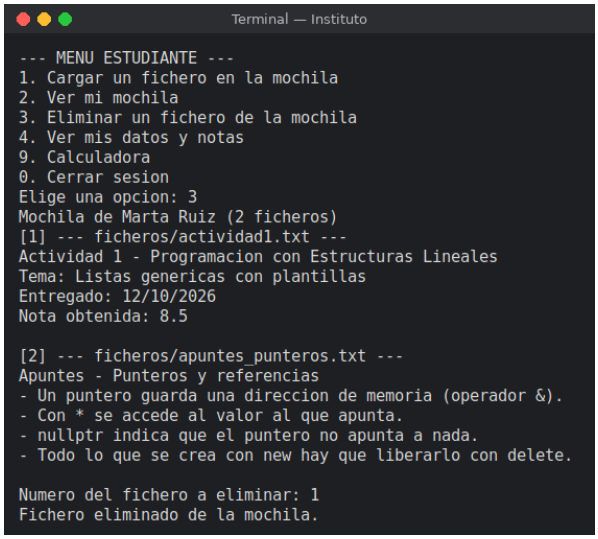


Figura 39. *Eliminar un documento de la mochila.*

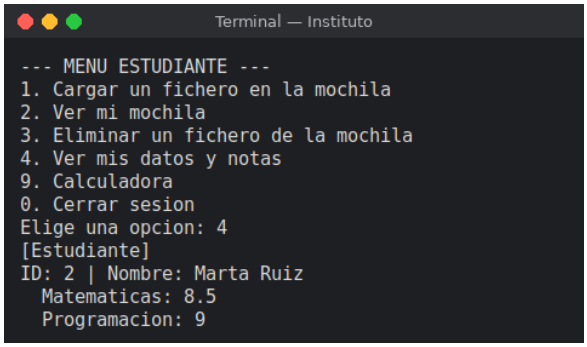


Figura 40. *Datos y notas de la estudiante.*

6.6. Calculadora

Disponible desde los tres menús con la opción 9. Las opciones 1 a 4 operan con dos números. La opción 5 opera con dos matrices: primero se elige la operación y después se introducen las filas, las columnas y los valores de cada matriz. Las opciones 6 y 7 calculan la traspuesta y el determinante de una matriz.

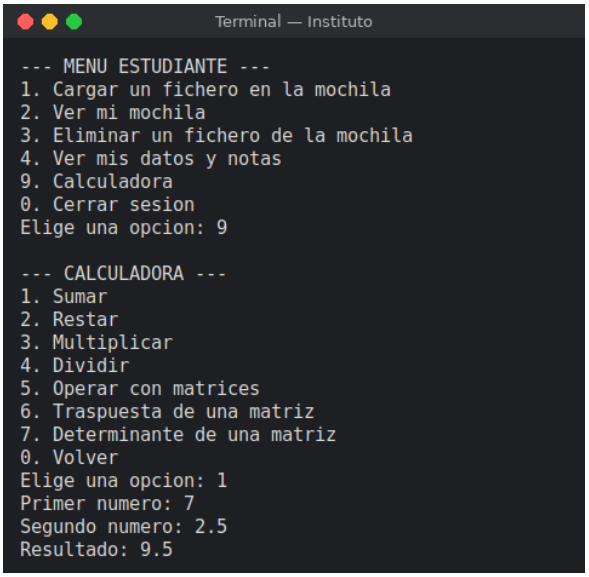


Figura 41. *Menú de la calculadora y una suma.*

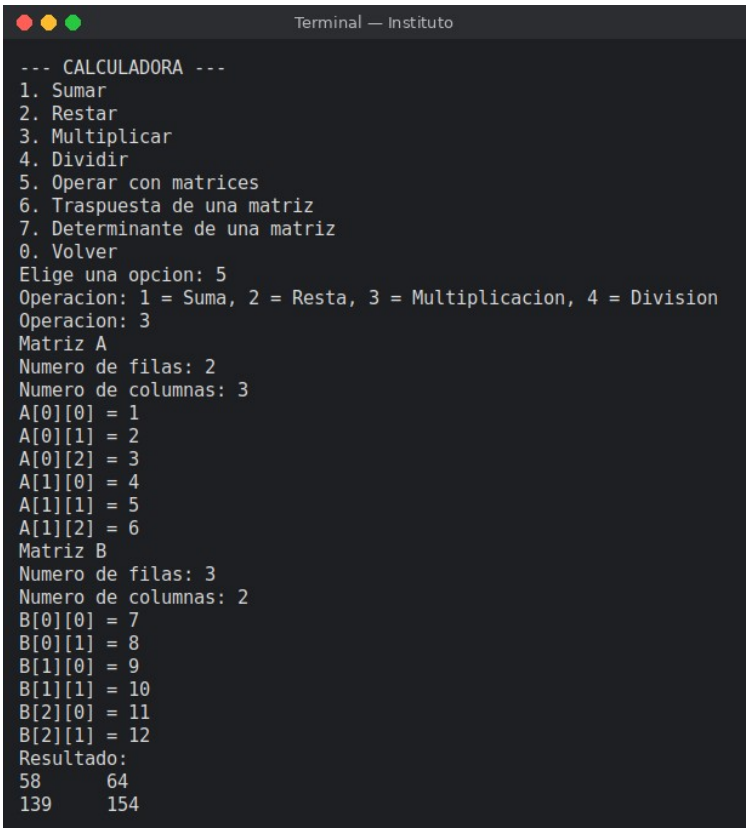


Figura 42. *Multiplicación de una matriz 2×3 por una 3×2.*

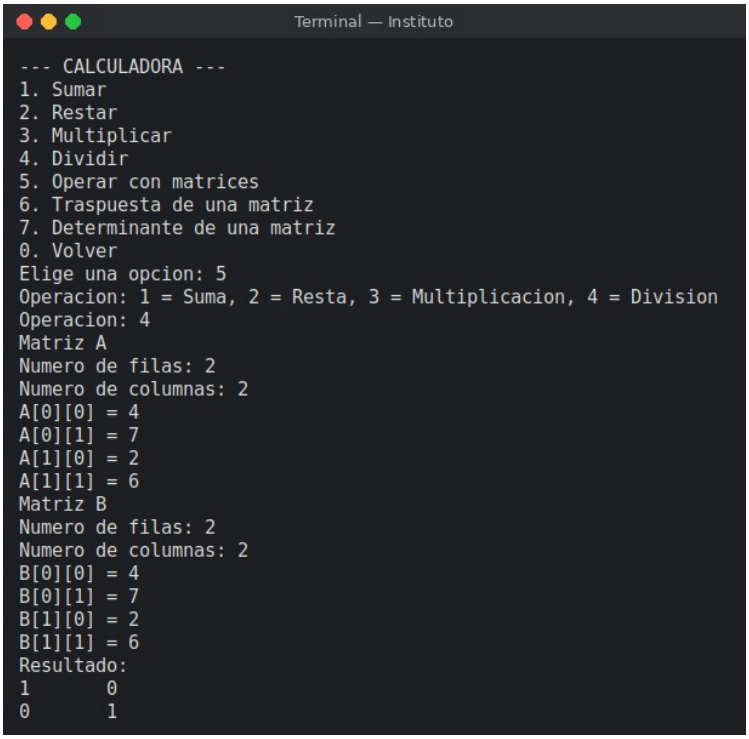


Figura 43. *División A / B = A · B^{-1}: al dividir una matriz entre sí misma se obtiene la identidad.*

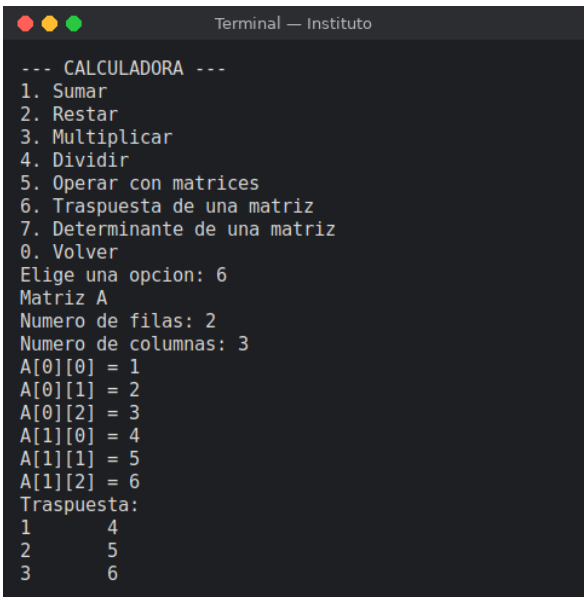


Figura 44. *Traspuesta de una matriz 2×3.*

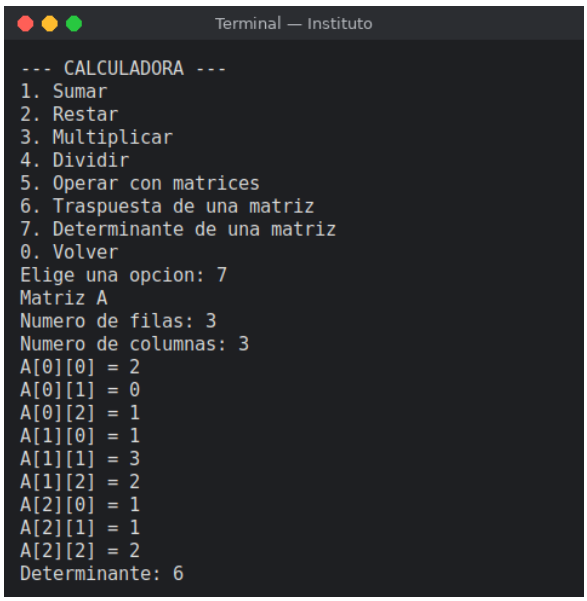


Figura 45. *Determinante de una matriz 3×3.*

6.7. Salir y límites del programa

Desde cualquier menú, `0` cierra la sesión; en la pantalla de inicio, `0` cierra el programa y libera toda la memoria (Figura 46).

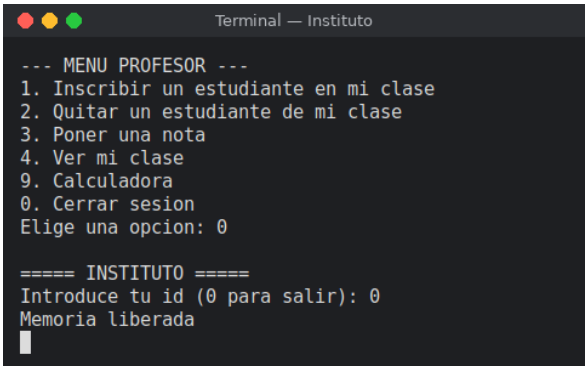


Figura 46. *Salida del programa.*

- Como máximo 50 usuarios, 50 estudiantes por clase, 50 asignaturas por estudiante y 50 documentos por mochila (tamaño de `ListaGenerica`).
- Matrices de 1×1 a 10×10.
- Los nombres, apellidos y asignaturas no pueden estar vacíos. Las opciones y los ids deben ser números: si se escribe otra cosa, el programa lo vuelve a pedir.

7. Pruebas

7.1. Casos de prueba

Todas las capturas de este documento son de ejecuciones reales del programa. Esta tabla resume los casos probados, incluidos los casos límite y de error:

#	Caso	Resultado esperado	Obtenido
1	Entrar con id 1	Menú de administrador	Correcto (Figura 27)
2	Alta de estudiante y profesor	Ids 2 y 3	Correcto (Figura 28)
3	Baja del id 4 y nueva alta	El nuevo usuario recibe el id 4	Correcto (Figura 30)
4	Modificar un id inexistente	«El id no existe»	Correcto (Figura 32)
5	El administrador se da de baja a sí mismo	No se permite	Correcto (Figura 47)
6	Letras en lugar de una opción	Se vuelve a pedir el número	Correcto (Figura 47)
7	Rol de usuario no válido	«Rol no valido»	Correcto (Figura 49)
8	Inscribir a un administrador / a un alumno que ya tiene profesor	Se rechaza con su mensaje	Correcto (Figura 48)
9	Nota 12	«La nota debe estar entre 0 y 10»	Correcto (Figura 50)
10	Nota o baja de un alumno que no es de la clase	«Estudiante no encontrado»	Correcto (Figura 51)
11	Cargar un fichero que no existe / una carpeta	Error controlado, la mochila no cambia	Correcto (Figura 52)
12	Eliminar un documento que no existe	«Posicion no valida»	Correcto (Figura 53)
13	División entre 0	«Division entre cero»	Correcto (Figura 54)
14	División por una matriz singular	«La matriz divisor no es invertible»	Correcto (Figura 55)
15	Determinante de una matriz no cuadrada / operación 9	Error controlado	Correcto (Figura 56)
16	Sumar matrices de distinto tamaño	«Las matrices deben tener las mismas dimensiones»	Correcto (Figura 57)
17	Baja de una alumna inscrita en una clase	Desaparece de la clase del profesor	Correcto (Figura 58, Figura 59)
18	Baja de un profesor	Sus alumnos quedan libres y su id se reutiliza	Correcto (Figura 60, Figura 61)
19	Ejecución completa con AddressSanitizer	Sin fugas ni accesos inválidos	Correcto (Figura 62)

7.2. Errores controlados

En ningún caso el programa se cierra: se muestra el error y se vuelve al menú.

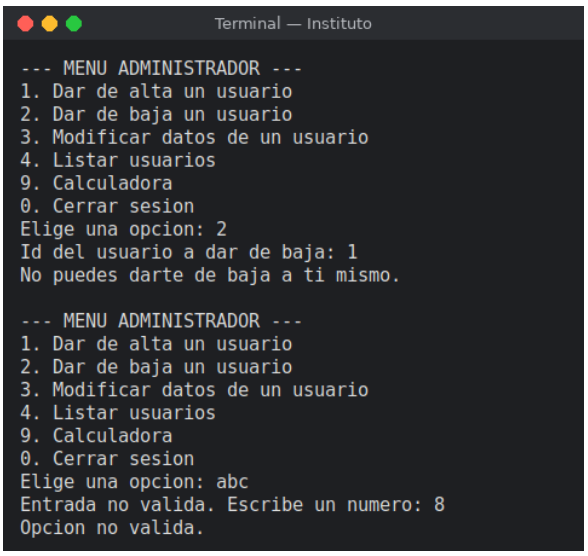


Figura 47. *El administrador no puede darse de baja; entrada no numérica y opción inexistente.*

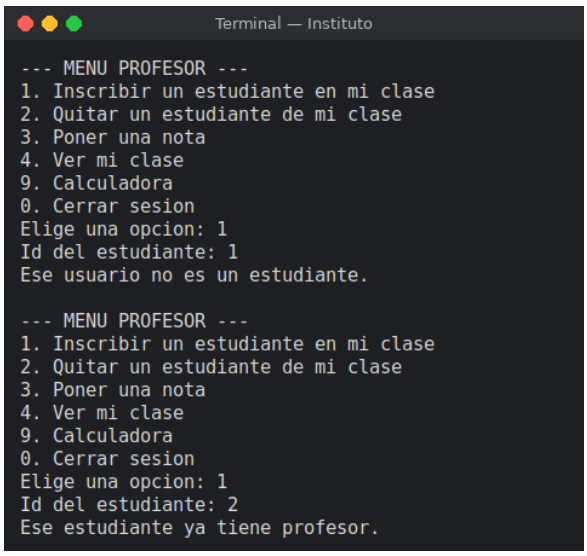


Figura 48. *Inscribir a un usuario que no es estudiante y a una alumna que ya tiene profesor.*

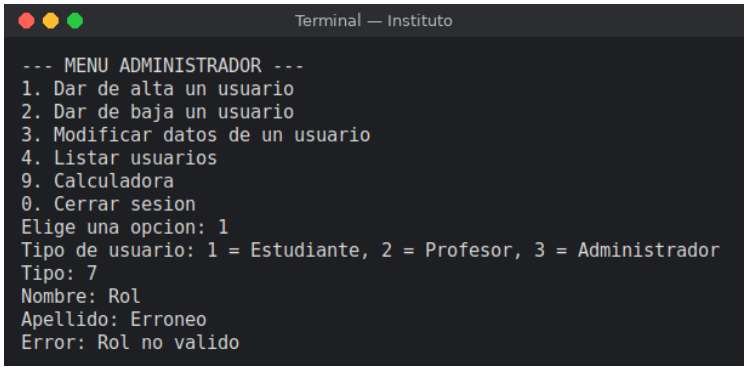


Figura 49. *Alta con un rol no válido.*

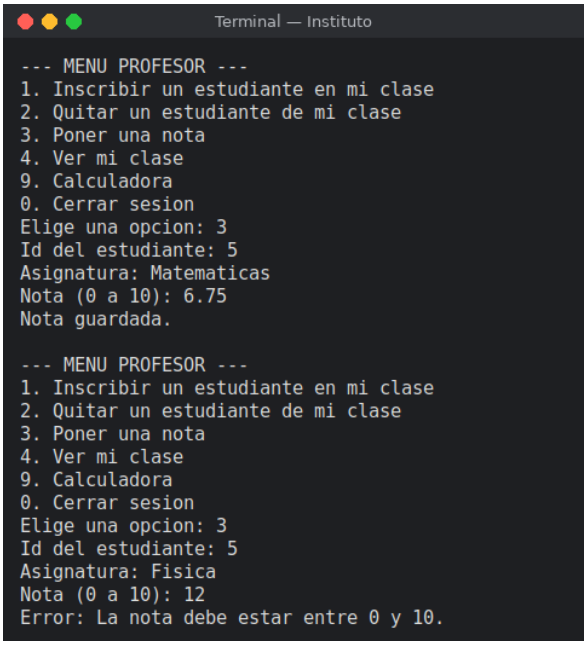


Figura 50. *Una nota válida y una fuera de rango.*

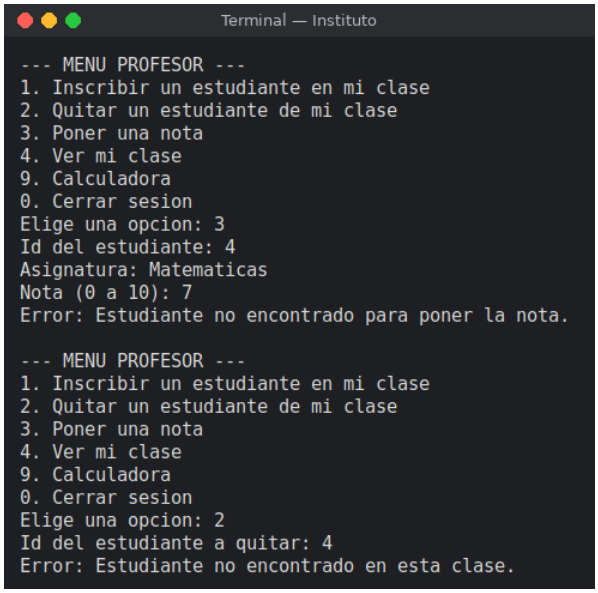


Figura 51. *Poner nota o quitar a un alumno que no está en la clase.*

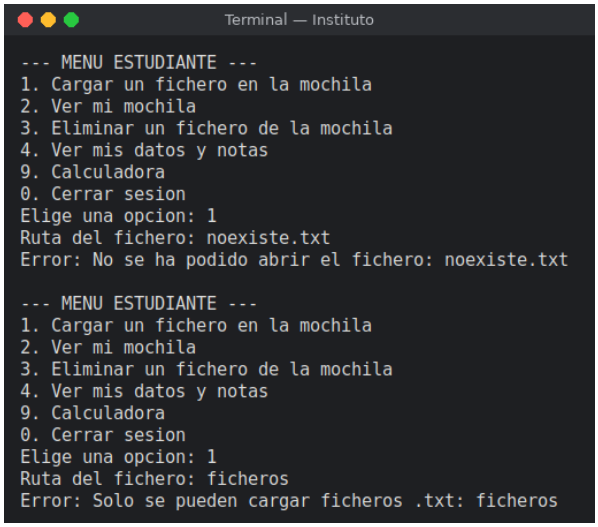


Figura 52. *Fichero inexistente y ruta que no es un .txt.*

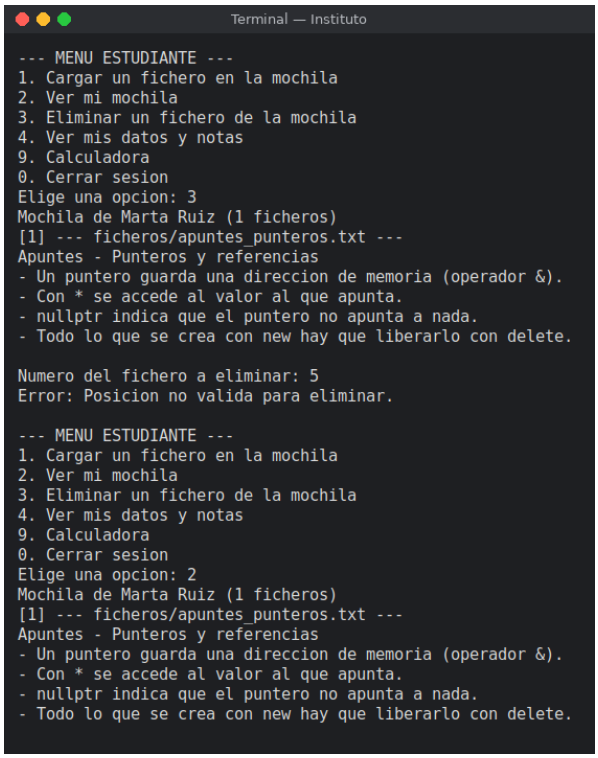


Figura 53. *Número de documento no válido y mochila tras eliminar un fichero.*

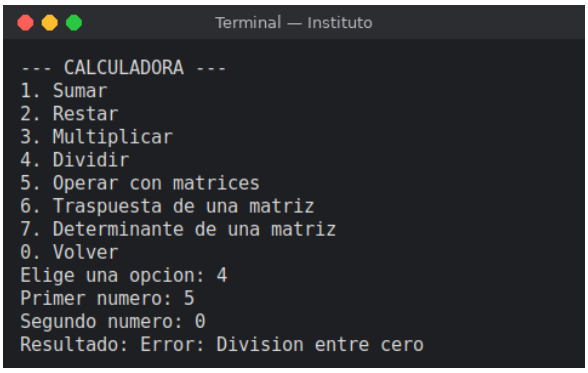


Figura 54. *División entre cero.*

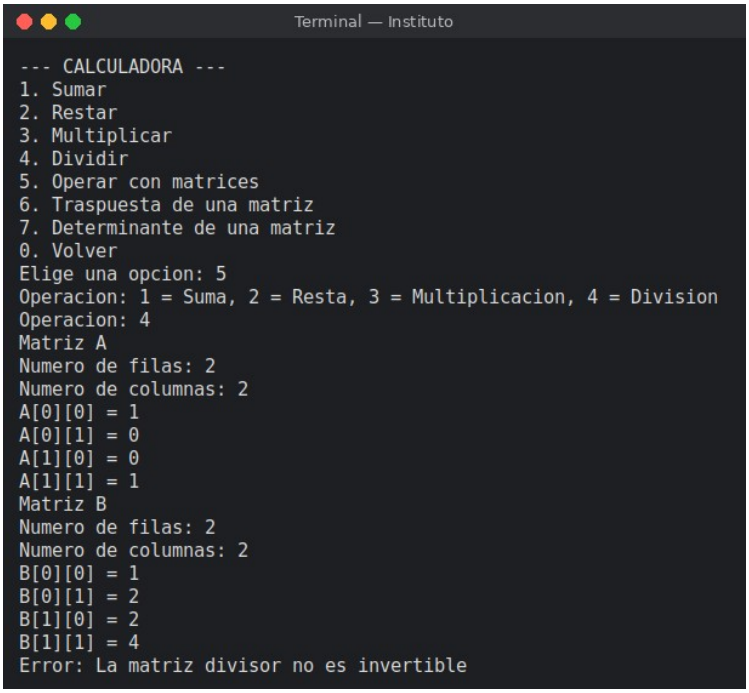


Figura 55. *División por una matriz que no es invertible.*

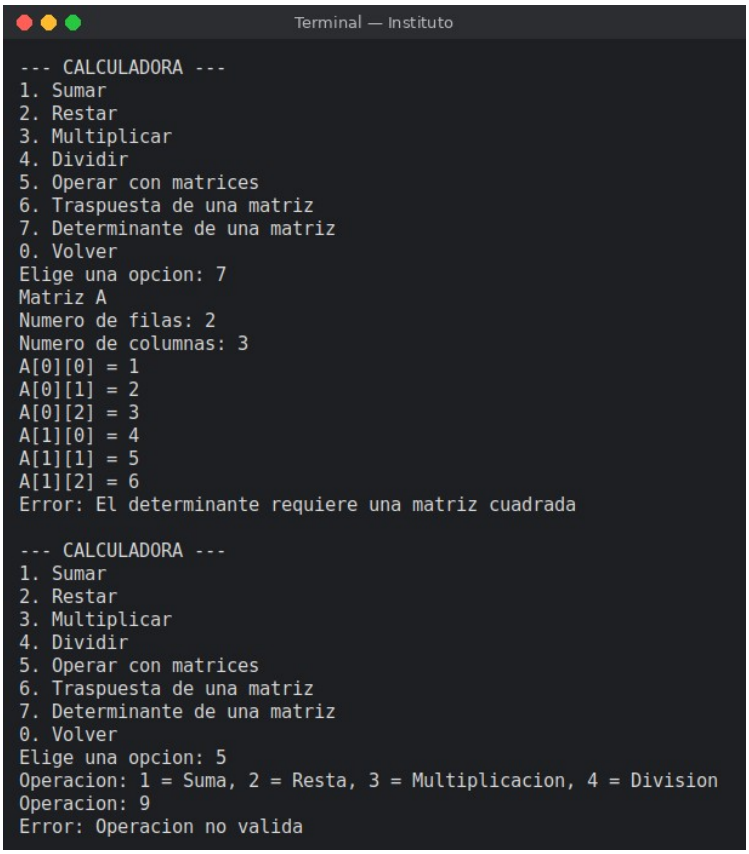


Figura 56. *Determinante de una matriz no cuadrada y operación no válida.*

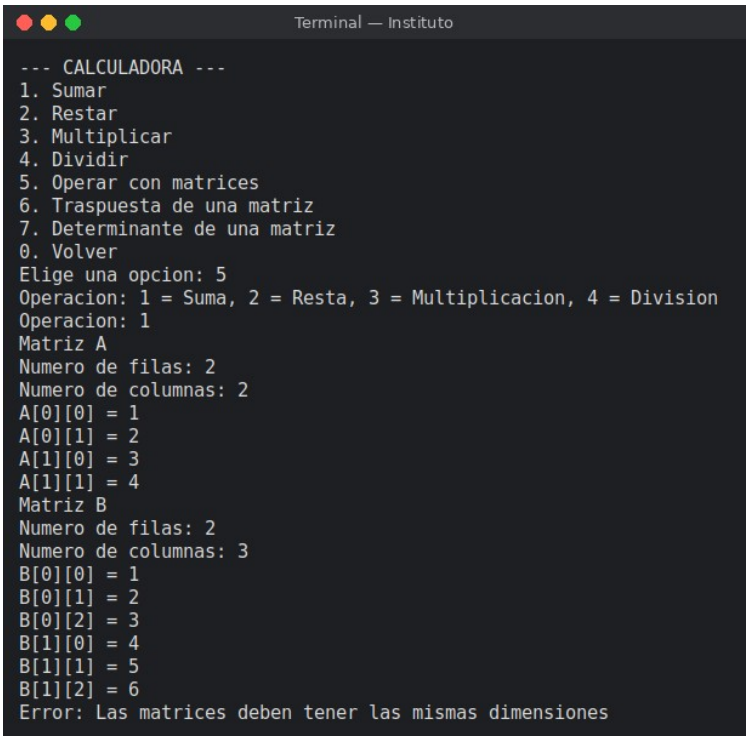


Figura 57. *Suma de matrices de dimensiones distintas.*

7.3. Bajas y destructores

Estas pruebas comprueban que los destructores deshacen las relaciones entre objetos. Primero se da de baja a Ana (id 5), que estaba inscrita en la clase de Juan (Figura 58). Al volver a ver la clase, Ana ya no aparece y quedan 1 inscrito (Figura 59): `~Estudiante()` la ha quitado de la lista del profesor. Sin ese destructor, la clase conservaría un puntero colgante y al mostrarla se accedería a memoria liberada.

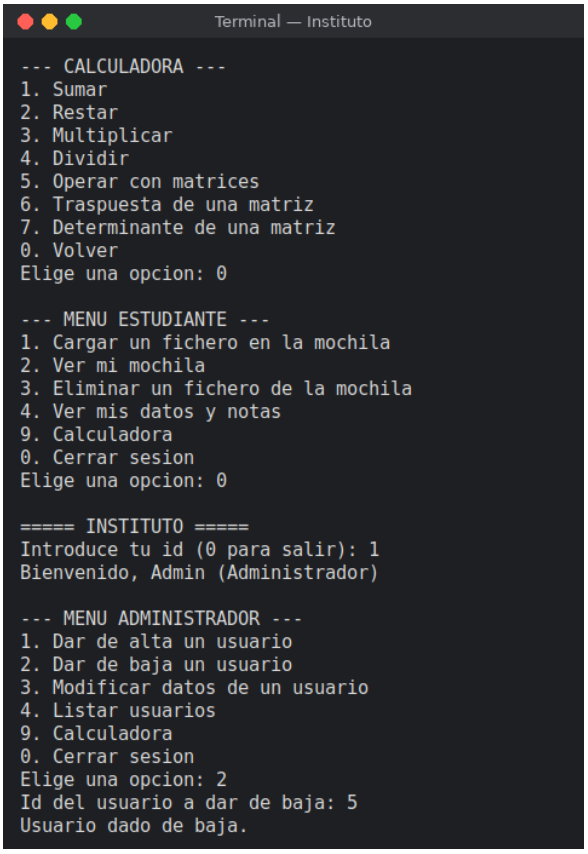


Figura 58. *Baja de una alumna inscrita en una clase.*

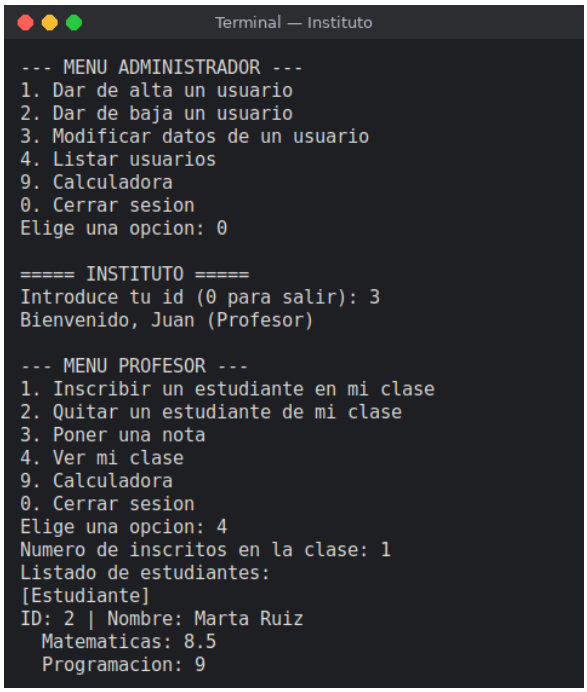


Figura 59. *La clase del profesor tras la baja: ya no aparece.*

Después se da de baja al profesor (id 3) y se da de alta a una profesora nueva, que recibe el id 3 (Figura 60). La nueva profesora puede inscribir a Marta (Figura 61), lo que demuestra que `~Profesor()` dejó a sus antiguos alumnos sin profesor.

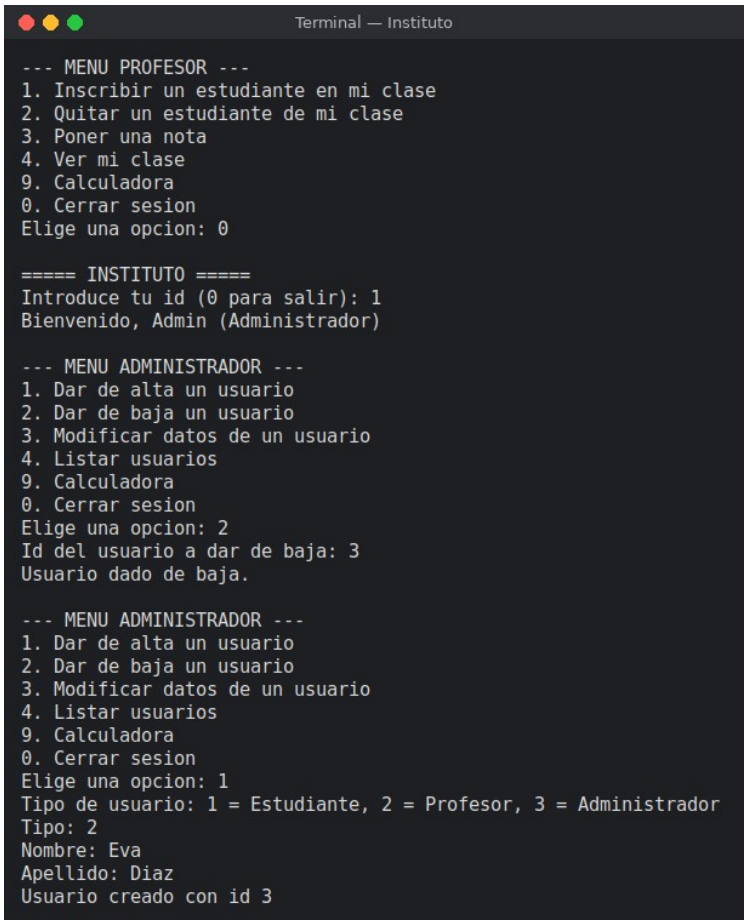


Figura 60. *Baja del profesor y alta de una profesora que reutiliza el id 3.*

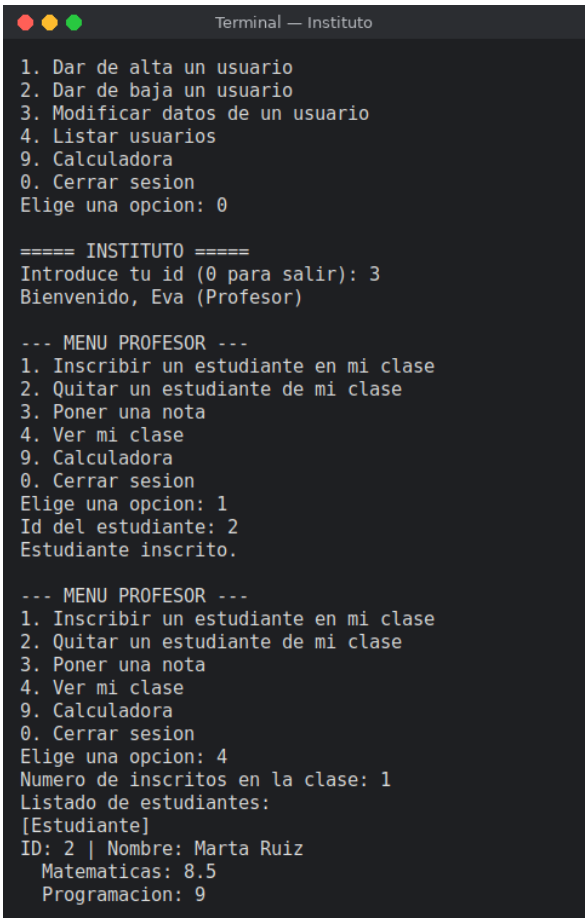


Figura 61. *La nueva profesora inscribe a la alumna que tenía el profesor dado de baja.*

7.4. Prueba de memoria con AddressSanitizer

Para comprobar que no hay fugas, dobles liberaciones ni accesos a memoria liberada, compilamos el programa con **AddressSanitizer** (`-fsanitize=address,undefined`). Es una herramienta de GCC y Clang que vigila cada acceso a memoria y, al terminar, informa de cualquier memoria dinámica que no se haya liberado. Ejecutamos un guion de prueba que recorre todas las opciones del programa (altas, bajas, notas, mochila y calculadora) y redirigimos los errores a un fichero.

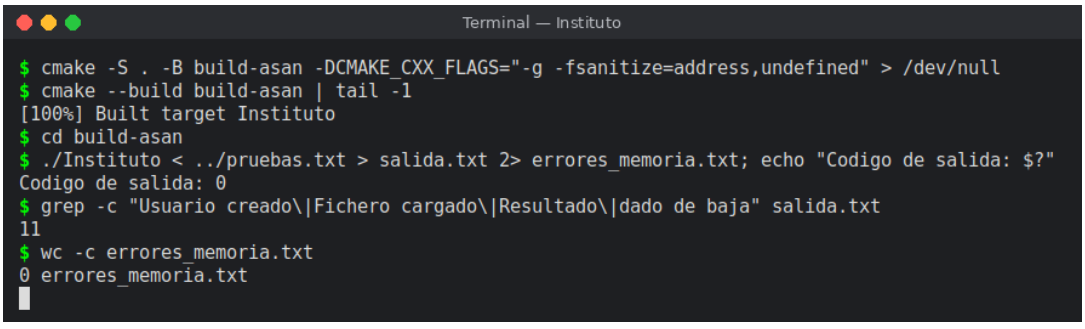


Figura 62. *Ejecución completa con AddressSanitizer: el fichero de errores queda vacío (0 bytes).*

El programa termina con código 0 y el fichero de errores queda **vacío**: no hay ninguna fuga ni ningún acceso inválido. Esta prueba nos sirvió para detectar errores durante el desarrollo, por ejemplo un puntero colgante cuando un alumno se daba de baja antes de existir los destructores.

8. Reparto del trabajo

Organizamos el trabajo en GitHub con un issue por parte del proyecto, una rama por issue y un *pull request* para unir cada parte a la rama principal. El reparto fue el siguiente:

Parte	Issue	Responsable
Base del proyecto: estructura, contrato de cabeceras, `ListaGenerica<T>` y `Usuario`	#1	Carlota Parra
Recurso 1: `Profesor` y gestión de la clase	#3	Carlota Parra
Recurso 2: `Estudiante`, notas por asignatura y mochila de `Fichero`	#4	Adrián Duque
Recurso 3: `Matriz<T>` y calculadora	#5	Ignacio del Peso
Recurso 4: `Administrador`, altas, bajas y modificación de usuarios	#2	Ignacio del Peso
`Main.cpp`: inicio de sesión, menús por rol y manejo de excepciones	#6	Sofía Peral
Memoria	#7	Adrián Duque
Pruebas finales, correcciones de memoria dinámica y entrega	#8	Adrián Duque

Antes de repartir el trabajo acordamos las cabeceras (`.h`) de todas las clases, para que cada persona pudiera programar su parte sin esperar a las demás. El diagrama UML se ha ido actualizando con los cambios del código.

9. Conclusiones

La actividad nos ha servido para entender en la práctica la idea central de la UF2: **quien adquiere un recurso tiene que garantizar su liberación**. En Java la memoria se libera sola; en C++ cada `new` necesita un propietario claro y un `delete` en el momento justo. Lo que más nos enseñó fue ver, con AddressSanitizer, cómo un programa que parecía funcionar bien accedía a memoria ya liberada al dar de baja a un alumno inscrito. Al moverse la limpieza a los destructores, el código de las bajas quedó más corto y más seguro.

También hemos visto la importancia de `virtual`. Sin él, guardar todos los usuarios como `Usuario*` no sirve de nada, porque siempre se ejecuta la versión de la clase base, y un `delete` a través de la base deja sin liberar lo que poseen las clases hijas.

La lista genérica de la UF1 ha sido la base de todo el proyecto: la misma plantilla guarda usuarios, alumnos, notas, documentos e ids, y así no hemos tenido que repetir código ni gestionar contadores a mano en cada clase.

9.1. Posibles mejoras

- Pasar los `std::string` de los constructores de `Usuario` y `Profesor` por referencia constante, como ya se hace en el resto de clases.
- Añadir a `ListaGenerica` operaciones para insertar o modificar en cualquier posición, que simplificarían `Estudiante::ponerNota`.
- Usar punteros inteligentes, que la UF2 menciona como forma de evitar fugas. En esta actividad hemos usado `new` y `delete` a propósito para practicar la gestión manual que pide la unidad.

10. Referencias

- García González, J. *Unidad Formativa 1: Tipos Abstractos de Datos y Principios de Diseño*. Programación con Estructuras Lineales, Universidad Europea.
- García González, J. *Unidad Formativa 2: Gestión de recursos*. Programación con Estructuras Lineales, Universidad Europea.
- Doval Abilleira, L. *Guía para programar en C++*. Programación con Estructuras Lineales, Universidad Europea.
- Ejercicios de clase: *Lista Genérica* y *MatrizGenerica*.
- cppreference.com: referencia del lenguaje C++ y de la biblioteca estándar (`std::exception`, `std::ifstream`).

Sistema de administración académico

Actividad Colaborativa 2 · Programación con Estructuras Lineales

Grupo 3

Adrián Duque · Carlota Parra · Ignacio del Peso · Sofía Peral

Universidad Europea

Grado en Ingeniería Informática · Curso 2026-2027